

IUPC / ICPC TEAM REFERENCE NOTEBOOK 2026

Jashore University of Science and Technology | Competitive Programming Reference Document

TABLE OF CONTENTS	
1. Setup, Fast I/O & Debugging	2
2. Mathematics & Basic Algebra	2
3. Number Theory & Divisors	3
4. Combinatorics & Discrete Math	5
5. Data Structures	6
6. String Algorithms	8
7. Graph Theory	9
8. Dynamic Programming	11
9. Greedy, Searching & Techniques	12
10. Computational Geometry	12
11. Bit Hacks, Game Theory & STL Reference	13

1. Setup, Fast I/O & Debugging

1.1 Contest Boilerplate (Marwan's Setup)

Standard fast competitive programming template with 64-bit int and helper macros.

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
#define M 1000000007
#define N 1000005
#define INF 1e17
#define endl "\n"
#define all(v) (v).begin(), (v).end()
#define yes cout << "Yes" << endl
#define no cout << "No" << endl
#define minus cout << "-1" << endl
#define zero cout << "0" << endl
#define make_unique(x) sort(all({x})); (x).erase(unique(all({x})), (x).end())
```

```
void marwan() {
    // Solve logic
}
```

```
int32_t main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    int t = 1;
    cin >> t;
    while (t--) marwan();
    return 0;
}
```

1.2 GCC Optimization Pragmas

Include at top of file for heavy CPU tasks. May disable vectorization on some platforms.

```
#pragma GCC optimize("O3,unroll-loops")
#pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt")
```

1.3 Sublime Text Build Systems

Automated compilation and local file redirection.

```
// Linux:
{
    "shell": true,
    "cmd": "g++ -std=c++17 '${file}' -o '${file_path}/${file_base_name}' && '${file_path}/${file_base_name}' < input.txt > output.txt",
    "working_dir": "${file_path}",
    "selector": "source.cpp"
}

// Windows:
{
    "cmd": ["g++.exe", "-std=c++17", "${file}", "-o", "${file_base_name}.exe", "&&", "${file_base_name}.exe<input.txt> output.txt"],
    "shell": true,
    "working_dir": "${file_path}",
    "selector": "source.cpp"
}
```

1.4 Fast I/O (fread based)

Reads integers ~ 5x faster than cin. Do NOT mix with cin/scanf.

```
inline char gc() {
    static char buf[1 << 16];
    static size_t bc = 0, be = 0;
    if (bc ≥ be) {
        buf[0] = 0; bc = 0;
        be = fread(buf, 1, sizeof(buf), stdin);
    }
    return buf[bc++];
}
```

```
}
int readInt() {
    int a, c;
    while ((a = gc()) < 40) if (a == 0) return 0;
    bool minus = (a == '-');
    if (minus) a = gc();
    int res = a - '0';
    while ((c = gc()) ≥ '0' && c ≤ '9') res = res * 10 + (c - '0');
    return minus ? -res : res;
}
```

1.5 Universal Debug Macro

Prints variable names and container contents (vector, set, map, pair) to stderr.

```
#ifndef ONLINE_JUDGE
#define dbg(...) cerr << __LINE__ << ": ", _dbg(#__VA_ARGS__, __VA_ARGS__), cerr << "\n"
template<class T> void _p(T x) { cerr << x; }
template<class T, class U> void _p(pair<T, U> x) { cerr << "(" << x.first << ", " << x.second << ")"; }
template<class T> void _p(vector<T> x) { cerr << "["; int _=0; for (auto y : x) { if (_++) cerr << ","; _p(y); } cerr << "]"; }
template<class T> void _p(set<T> x) { cerr << "{"; int _=0; for (auto y : x) { if (_++) cerr << ","; _p(y); } cerr << "}"; }
template<class T, class U> void _p(map<T, U> x) { cerr << "["; int _=0; for (auto [k, v] : x) { if (_++) cerr << ","; _p(k); cerr << "; "; _p(v); } cerr << "]"; }
void _dbg(const char* s) {}
template<class T, class... U> void _dbg(const char* s, T x, U... t) {
    int i = 0; while (s[i] && s[i] ≠ ',') i++;
    cerr.write(s, i) << "="; _p(x);
    if constexpr (sizeof...(t)) cerr << " || ", _dbg(s + i + 1, t...);
}
#else
#define dbg(...)
#endif
```

1.6 GP Hash Table & Anti-Hash Test

High-performance hash table immune to worst-case anti-hash tests.

```
#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;
```

```
struct custom_hash {
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }
    size_t operator()(uint64_t x) const {
        static const uint64_t FIXED_RANDOM = chrono::steady_clock::now().time_since_epoch().count();
        return splitmix64(x + FIXED_RANDOM);
    }
    size_t operator()(pair<int, int> p) const noexcept {
        return (*this)(p.first) ^ ((*this)(p.second) << 1);
    }
};
template<typename K, typename V>
using safe_map = gp_hash_table<K, V, custom_hash>;
template<typename K>
using safe_set = gp_hash_table<K, null_type, custom_hash>;
```

1.7 Basic Node Structures (Linked List & Tree)

Snippet definitions for pointers, linked lists, and tree nodes.

```
class Node {
public:
    int data;
    Node* next;
```

```
Node(int d) : data(d), next(nullptr) {}
};
```

```
class TreeNode {
public:
    int val;
    TreeNode *left, *right;
    TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
};
```

1.8 Contest Complexity & Constants Cheat Sheet

Rules of thumb for 1.0 second time limits (≈ 10⁸ operations).

- $N \leq 10$: $O(N!)$ ≈ 3.6 × 10⁶
- $N \leq 20$: $O(2^N)$ ≈ 10⁶
- $N \leq 500$: $O(N^3)$ ≈ 1.25 × 10⁸
- $N \leq 5000$: $O(N^2)$ ≈ 2.5 × 10⁷
- $N \leq 10^6$: $O(N \log N)$ ≈ 2 × 10⁷
- $N \leq 10^8$: $O(N)$ linear
- $2^{10} \approx 10^3$, $2^{20} \approx 10^6$, $2^{30} \approx 10^9$, $2^{60} \approx 10^{18}$

2. Mathematics & Basic Algebra

2.1 Series & Sums Formulas

Closed form expressions for standard summations and power series.

- Arithmetic Progression: $S_n = \frac{n}{2}(2a + (n-1)d) = \frac{n}{2}(a_1 + a_n)$
- Geometric Progression: $\sum_{i=0}^n c^i = \frac{c^{n+1}-1}{c-1}$, $\sum_{i=a}^b c^i = \frac{c^{b+1}-c^a}{c-1}$ ($c \neq 1$)
- Infinite GP: $\sum_{i=0}^\infty c^i = \frac{1}{1-c}$ ($|c| < 1$)
- Sum of first n integers: $\sum_{i=1}^n i = \frac{n(n+1)}{2}$
- Sum of squares: $\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$
- Sum of cubes: $\sum_{i=1}^n i^3 = \left[\frac{n(n+1)}{2}\right]^2$
- Sum of 4th powers: $\sum_{i=1}^n i^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$
- Harmonic Numbers: $H_n = \sum_{i=1}^n \frac{1}{i} \approx \ln n + \gamma$ ($\gamma \approx 0.57721566$)
- Weighted Harmonic: $\sum_{i=1}^n iH_i = \frac{n(n+1)}{2}H_n - \frac{n(n-1)}{4}$

2.2 Algebraic Identities

Factorizations and expansions frequently tested in algebraic manipulations.

- $(a \pm b)^2 = a^2 \pm 2ab + b^2$ | $a^2 - b^2 = (a - b)(a + b)$
- $(a \pm b)^3 = a^3 \pm 3a^2b + 3ab^2 \pm b^3$ | $a^3 \pm b^3 = (a \pm b)(a^2 \mp ab + b^2)$
- $(a + b + c)^2 = a^2 + b^2 + c^2 + 2(ab + bc + ca)$
- $a^4 + 4b^4 = (a^2 - 2ab + 2b^2)(a^2 + 2ab + 2b^2)$ (Sophie Germain)
- $a^n - b^n = (a - b)(a^{n-1} + a^{n-2}b + \dots + b^{n-1})$
- $a^n + b^n = (a + b)(a^{n-1} - a^{n-2}b + \dots + b^{n-1})$ (for odd n)

2.3 Classical Inequalities

Inequalities used for bounds, binary searches, and proofs.

- AM-GM: $\frac{x_1+x_2+\dots+x_n}{n} \geq \sqrt[n]{x_1x_2\dots x_n}$ (equality iff all x_i equal)
- Cauchy-Schwarz: $(\sum a_i^2)(\sum b_i^2) \geq (\sum a_ib_i)^2$
- Triangle: $||a| - |b|| \leq |a \pm b| \leq |a| + |b|$
- Jensen's: $f\left(\frac{\sum x_i}{n}\right) \leq \frac{\sum f(x_i)}{n}$ for convex f ($f'' \geq 0$)
- Bernoulli: $(1+x)^n \geq 1+nx$ for $n \geq 0, x > -1$
- Chebyshev: $\frac{1}{n} \sum a_ib_i \geq (\frac{1}{n} \sum a_i)(\frac{1}{n} \sum b_i)$ if sorted similarly

2.4 Probability & Expected Value

Rules of probability, linearity of expectation, and statistical bounds.

- $P(A \cup B) = P(A) + P(B) - P(A \cap B)$
- Bayes' Theorem: $P(A|B) = \frac{P(B|A)P(A)}{P(B)}$
- Linearity of Expectation: $E[X + Y] = E[X] + E[Y]$ (ALWAYS holds, even if X, Y are dependent!)
- Independent Variables: $E[X \cdot Y] = E[X] \cdot E[Y]$
- Variance: $\text{Var}(X) = E[X^2] - (E[X])^2$, $\text{Var}(aX + b) = a^2 \text{Var}(X)$
- Markov: $P(X \geq a) \leq \frac{E[X]}{a}$ ($X \geq 0, a > 0$)
- Chebyshev: $P(|X - \mu| \geq k\sigma) \leq \frac{1}{k^2}$

2.5 Binary Exponentiation

Computes $a^b \pmod{m}$ in $O(\log b)$ time.

```
int binexp(int a, int b, int m = M) {
    int res = 1 % m; a %= m;
    while (b > 0) {
        if (b & 1) res = (__int128)res * a % m;
        a = (__int128)a * a % m;
        b >>= 1;
    }
    return res;
}
```

2.6 Matrix Exponentiation

Solves linear recurrences and counts graph paths of length K in $O(N^3 \log K)$.

```
template<size_t N>
struct Matrix {
    int mat[N][N];
    Matrix(int v = 0) {
        for (size_t i = 0; i < N; i++)
            for (size_t j = 0; j < N; j++)
                mat[i][j] = (i == j ? v : 0);
    }
    Matrix operator*(const Matrix& other) const {
        Matrix res(0);
        for (size_t i = 0; i < N; i++) {
            for (size_t k = 0; k < N; k++) {
                if (!mat[i][k]) continue;
                for (size_t j = 0; j < N; j++) {
                    res.mat[i][j] = (res.mat[i][j] + (__int128)mat[i][k] *
                        other.mat[k][j]) % M;
                }
            }
        }
        return res;
    }
};
template<size_t N>
```

```
Matrix<N> mat_expo(Matrix<N> a, int p) {
    Matrix<N> res(1);
    while (p > 0) {
        if (p & 1) res = res * a;
        a = a * a;
        p >>= 1;
    }
    return res;
}
// Fibonacci: F[n] = F[n-1] + F[n-2], F[0]=0, F[1]=1
int fib_mat(int n) {
    if (n == 0) return 0;
    Matrix<2> T;
    T.mat[0][0] = 1; T.mat[0][1] = 1;
    T.mat[1][0] = 1; T.mat[1][1] = 0;
    T = mat_expo(T, n - 1);
    return T.mat[0][0]; // F[n]
}
```

2.7 Basis Vector (XOR Basis)

Maintains linear basis of vectors over $\mathbb{F}_2$. Supports max XOR subset, k-th XOR.

```
template<typename T = int, int LG = 60>
struct Basis {
    T basis[LG];
    int sz;
    Basis() { reset(); }
    void reset() { fill(basis, basis + LG, 0); sz = 0; }
    bool insert(T mask) {
        for (int i = LG - 1; i >= 0; --i) {
            if ((mask >> i) & 1) {
                if (basis[i]) mask ^= basis[i];
                else { basis[i] = mask; ++sz; return true; }
            }
        }
        return false;
    }
    T maxXor() {
        T ans = 0;
        for (int i = LG - 1; i >= 0; --i) ans = max(ans, ans ^ basis[i]);
        return ans;
    }
    bool can(T x) {
        for (int i = LG - 1; i >= 0; --i) {
            if ((x >> i) & 1) {
                if (!basis[i]) return false;
                x ^= basis[i];
            }
        }
        return true;
    }
    T kth(T k) { // 1-indexed k-th smallest XOR
        if (k < 1 || k > (1ULL << sz)) return -1;
        T mask = 0, tot = 1ULL << sz;
        for (int i = LG - 1; i >= 0; --i) {
            if (basis[i]) {
                T low = tot / 2;
                if ((low < k && !((mask >> i) & 1)) || (low >= k && ((mask >> i) & 1)))
                    mask ^= basis[i];
                if (low < k) k -= low;
                tot /= 2;
            }
        }
        return mask;
    }
};
```

2.8 Linear Combination Checking (Bézout's)

Check if integer N can be formed by $a \cdot x + b \cdot y + c \cdot z + \dots$

- N can be represented iff $N \pmod{\gcd(a_1, a_2, \dots, a_k)} = 0$.
- Frobenius Coin Problem: For 2 coprime integers a, b , the largest number that CANNOT be formed is $a \cdot b - a - b$. Exactly $\frac{(a-1)(b-1)}{2}$ numbers cannot be formed.

3. Number Theory & Divisors

3.1 GCD & LCM Fundamental Properties

Mathematical identities connecting GCD, LCM, divisibility, and lattice lines.

- $\gcd(a, b) \times \text{lcm}(a, b) = |a \cdot b|$
- $\gcd(a, b) = \gcd(a, b \pmod a) = \gcd(a, b - a)$
- $\gcd(k \cdot a, k \cdot b) = k \cdot \gcd(a, b)$
- $\gcd(a, b) = 1 \iff \gcd(a + b, a \cdot b) = 1$
- If $a \mid (b \cdot c)$ and $\gcd(a, b) = d$, then $\frac{a}{d} \mid c$.
- Exponents: $\gcd(a^m - 1, a^n - 1) = a^{\gcd(m, n)} - 1$
- Fibonacci GCD: $\gcd(F_m, F_n) = F_{\gcd(m, n)}$
- Lattice segment: The number of strict lattice points between $(0, 0)$ and (a, b) is $\gcd(|a|, |b|) - 1$.

3.2 Extended Euclidean Algorithm & Diophantine

Solves $a \cdot x + b \cdot y = \gcd(a, b)$ and linear Diophantine $a \cdot x + b \cdot y = c$.

```
int extgcd(int a, int b, int &x, int &y) {
    if (b == 0) { x = 1; y = 0; return a; }
    int x1, y1;
    int d = extgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}
bool find_any_solution(int a, int b, int c, int &x0, int &y0, int &g) {
    g = extgcd(abs(a), abs(b), x0, y0);
    if (c % g != 0) return false;
    x0 *= c / g; y0 *= c / g;
    if (a < 0) x0 = -x0;
    if (b < 0) y0 = -y0;
    // All solutions: x = x0 + k*(b/g), y = y0 - k*(a/g)
    return true;
}
```

3.3 Modular Inverse

Fermat's theorem for prime mod M ; Extended Euclidean for arbitrary coprime m .

```
// Prime mod: a^(m-2) mod m
int modInvPrime(int a, int m = M) { return binexp(a, m - 2, m); }

// Arbitrary mod (requires gcd(a, m) == 1):
int modInv(int a, int m) {
    int x, y;
    int g = extgcd(a, m, x, y);
    if (g != 1) return -1; // No inverse exists
    return (x % m + m) % m;
}
```

3.4 Chinese Remainder Theorem (General CRT)

Merges $x \equiv a_1 \pmod{m_1}$ and $x \equiv a_2 \pmod{m_2}$. Supports non-coprime moduli.

```
pair<int, int> CRT(int a1, int m1, int a2, int m2) {
    int x, y;
    int g = extgcd(m1, m2, x, y);
    if ((a2 - a1) % g != 0) return {-1, -1}; // No solution
}
```

```
int lcm = (m1 / g) * m2;
int t = ((__int128)(a2 - a1) / g * x) % (m2 / g);
int ans = (a1 + (__int128)m1 * t) % lcm;
if (ans < 0) ans += lcm;
return {ans, lcm}; // x === ans (mod lcm)
}
```

3.5 Sieve of Eratosthenes

Finds all primes up to N in $O(N \log \log N)$.

```
vector<int> primes;
vector<bool> is_prime(N + 1, true);
void sieve() {
    is_prime[0] = is_prime[1] = false;
    for (int i = 2; i * i ≤ N; i++) {
        if (is_prime[i]) {
            for (int j = i * i; j ≤ N; j += i) is_prime[j] = false;
        }
    }
    for (int i = 2; i ≤ N; i++) if (is_prime[i]) primes.push_back(i);
}
```

3.6 Smallest Prime Factor (SPF) & Linear Sieve

Computes SPF and primes in $O(N)$ time. Enables $O(\log N)$ factorization.

```
vector<int> spf(N + 1), primes;
void linear_sieve() {
    for (int i = 2; i ≤ N; i++) {
        if (!spf[i]) { spf[i] = i; primes.push_back(i); }
        for (int p : primes) {
            if (p > spf[i] || i * p > N) break;
            spf[i * p] = p;
        }
    }
}
// Factorization via SPF in  $O(\log N)$ :
vector<pair<int, int>> factorize_spf(int n) {
    vector<pair<int, int>> factors;
    while (n > 1) {
        int p = spf[n], c = 0;
        while (n % p == 0) { n /= p; c++; }
        factors.push_back({p, c});
    }
    return factors;
}
```

3.7 Prime Factorization in $O(\sqrt{N})$

Standard factorization for isolated large numbers up to 10^{14} .

```
vector<pair<int, int>> factorize(int n) {
    vector<pair<int, int>> factors;
    for (int d = 2; d * d ≤ n; d++) {
        if (n % d == 0) {
            int count = 0;
            while (n % d == 0) { n /= d; count++; }
            factors.push_back({d, count});
        }
    }
    if (n > 1) factors.push_back({n, 1});
    return factors;
}
```

3.8 Segmented Sieve

Finds all primes in range $[L, R]$ where $R \leq 10^{12}$, $R - L \leq 10^7$.

```
vector<int> segmented_sieve(int L, int R) {
    int lim = sqrt(R);
    vector<bool> mark(lim + 1, false);
    vector<int> small_primes;
    for (int i = 2; i ≤ lim; i++) {
        if (!mark[i]) {
            small_primes.push_back(i);
            for (int j = i * i; j ≤ lim; j += i) mark[j] = true;
        }
    }
}
```

```
}
vector<bool> isPrime(R - L + 1, true);
for (int p : small_primes) {
    int sm = (L + p - 1) / p * p;
    for (int j = max(p * p, sm); j ≤ R; j += p) isPrime[j - L] = false;
}
if (L ≤ 1) for (int i = L; i ≤ min(1LL, R); i++) isPrime[i - L] = false;
vector<int> res;
for (int i = 0; i ≤ R - L; i++) if (isPrime[i]) res.push_back(L + i);
return res;
}
```

3.9 Divisor Formulas (Complete Theory & Bounds)

All formulas for prime factorization $n = p_1^{a_1} p_2^{a_2} \dots p_k^{a_k}$.

- Number of Divisors: $\tau(n) = d(n) = \prod_{i=1}^k (a_i + 1)$
- Sum of Divisors: $\sigma(n) = \prod_{i=1}^k \frac{p_i^{a_i+1} - 1}{p_i - 1}$
- Product of Divisors: $\prod_{d|n} d = n^{\tau(n)/2}$
- Odd / Even Divisors: If $n = 2^a \cdot m$ (m odd), then number of odd divisors is $\tau(m)$, and number of even divisors is $a \cdot \tau(m)$.
- Sum of Powers of Divisors: $\sigma_k(n) = \prod_{i=1}^k \frac{p_i^{(a_i+1)k} - 1}{p_i^k - 1}$
- Dirichlet Hyperbola Method: $\sum_{i=1}^N d(i) = \sum_{i=1}^N \lfloor N/i \rfloor = 2 \sum_{i=1}^{\lfloor \sqrt{N} \rfloor} \lfloor N/i \rfloor - \lfloor \sqrt{N} \rfloor^2 \quad (O(\sqrt{N}))$
- Max Number of Divisors Bounds (Highly Composite Numbers):
 - $N \leq 10^3$: $\max d(n) = 32$ ($n = 840$)
 - $N \leq 10^6$: $\max d(n) = 240$ ($n = 720720$)
 - $N \leq 10^9$: $\max d(n) = 1344$ ($n = 735134400$)
 - $N \leq 10^{12}$: $\max d(n) = 6720$ ($n = 963761198400$)
 - $N \leq 10^{18}$: $\max d(n) = 103680 \dots 4.35 \times 10^7$

3.10 Sum of Floor(N/i) in $O(\sqrt{N})$

Groups equal values of $\lfloor N/i \rfloor$ into blocks $[l, r]$.

```
int sum_floor(int n) {
    int ans = 0;
    for (int l = 1, r; l ≤ n; l = r + 1) {
        r = n / (n / l);
        ans += (r - l + 1) * (n / l);
    }
    return ans;
}
```

3.11 Euler's Totient Function (ETF / $\phi(n)$)

Counts integers in $[1, n]$ coprime to n . $\phi(n) = n \prod_{p|n} (1 - 1/p)$.

```
//  $O(\sqrt{n})$  for single query:
int phi(int n) {
    int res = n;
    for (int i = 2; i * i ≤ n; i++) {
        if (n % i == 0) {
            while (n % i == 0) n /= i;
            res -= res / i;
        }
    }
}
```

```
if (n > 1) res -= res / n;
return res;
}
// Sieve for all phi[1..N] in  $O(N)$ :
vector<int> phi_arr(N + 1);
void sieve_phi() {
    iota(phi_arr.begin(), phi_arr.end(), 0);
    for (int i = 2; i ≤ N; i++) {
        if (phi_arr[i] == i) {
            for (int j = i; j ≤ N; j += i) phi_arr[j] -= phi_arr[j] / i;
        }
    }
}
ETF Key Properties:  $\sum_{d|n} \phi(d) = n$ ; Sum of coprimes  $\sum_{1 \leq i \leq n, \gcd(i,n)=1} i = \frac{n \cdot \phi(n)}{2}$ . Euler's Totient Theorem:  $a^{\phi(m)} \equiv 1 \pmod m$  for  $\gcd(a, m) = 1$ . General Power Tower Reduction:  $a^b \equiv a^{(b \bmod \phi(m)) + \phi(m)} \pmod m$  for  $b \geq \phi(m)$ .
```

3.12 Miller-Rabin Primality Test (Deterministic 64-bit)

Checks if $N < 2^{64}$ is prime in $O(\log^3 N)$ using deterministic witness bases.

```
using u64 = uint64_t;
using u128 = __uint128_t;
u64 bin_pow(u64 a, u64 d, u64 mod) {
    u64 res = 1; a %= mod;
    while (d > 0) {
        if (d & 1) res = (u128)res * a % mod;
        a = (u128)a * a % mod;
        d >>= 1;
    }
    return res;
}
bool miller_rabin(u64 n) {
    if (n < 2) return false;
    if (n == 2 || n == 3) return true;
    if (n % 2 == 0) return false;
    u64 d = n - 1; int s = 0;
    while ((d & 1) == 0) { d >>= 1; s++; }
    static const u64 bases[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022};
    for (u64 a : bases) {
        if (a % n == 0) continue;
        u64 x = bin_pow(a, d, n);
        if (x == 1 || x == n - 1) continue;
        bool composite = true;
        for (int r = 1; r < s; r++) {
            x = (u128)x * x % n;
            if (x == n - 1) { composite = false; break; }
        }
        if (composite) return false;
    }
    return true;
}
```

3.13 Pollard's Rho (Fast 64-bit Factorization)

Finds a non-trivial factor of N in expected $O(N^{1/4})$ time.

```
int pollard_rho(int n) {
    if (n % 2 == 0) return 2;
    int x = 2, y = 2, d = 1, c = 1;
    auto f = [&](int val) { return ((__int128)val * val + c) % n; };
    while (d == 1) {
        x = f(x); y = f(f(y));
        d = std::gcd(abs(x - y), n);
        if (d == n) { x = rand() % (n - 2) + 2; y = x; c = rand() % (n - 1) + 1; d = 1; }
    }
    return d;
}
```

3.14 Floor Sum (AtCoder ACL)

Computes $\sum_{i=0}^{n-1} \lfloor \frac{a \cdot i + b}{m} \rfloor$ in $O(\log m)$ via Euclidean division.

```
int floor_sum(int n, int m, int a, int b) {
    int ans = 0;
    if (a ≥ m) { ans += (n - 1) * n * (a / m) / 2; a %= m; }
    if (b ≥ m) { ans += n * (b / m); b %= m; }
    int y_max = (a * n + b) / m;
    if (y_max == 0) return ans;
    ans += floor_sum(y_max, a, m, (a * n + b) % m);
    return ans;
}
```

3.15 Discrete Logarithm (Baby-Step Giant-Step)

Finds smallest $x \geq 0$ such that $a^x \equiv b \pmod m$ with $\gcd(a, m) = 1$ in $O(\sqrt{m})$.

```
int baby_step_giant_step(int a, int b, int m) {
    a %= m; b %= m;
    if (b == 1 || m == 1) return 0;
    int n = sqrt(m) + 1;
    unordered_map<int, int> tbl;
    int cur = b;
    for (int q = 0; q < n; q++) {
        tbl[cur] = q;
        cur = (__int128)cur * a % m;
    }
    int an = binexp(a, n, m);
    cur = 1;
    for (int p = 1; p ≤ n + 1; p++) {
        cur = (__int128)cur * an % m;
        if (tbl.count(cur)) {
            int ans = p * n - tbl[cur];
            return ans;
        }
    }
    return -1;
}
```

3.16 Möbius Inversion Formula

$\mu(n) = (-1)^k$ if n is square-free with k prime factors, 0 if $p^2 \mid n$.

- $\sum_{d|n} \mu(d) = [n == 1]$
- Inversion: $g(n) = \sum_{d|n} f(d) \iff f(n) = \sum_{d|n} \mu(d)g(n/d)$
- Co-primality count: $[\gcd(i, j) == 1] = \sum_{d|\gcd(i, j)} \mu(d)$
- GCD pairs: $\sum_{1 \leq i, j \leq N} [\gcd(i, j) = 1] = \sum_{d=1}^N \mu(d) \lfloor N/d \rfloor^2$

3.17 Lifting The Exponent (LTE)

$\nu_p(x)$ is exponent of highest power of prime p dividing x .

- Odd Prime ($p \geq 3$), $p \mid (a - b)$, $p \nmid a, p \nmid b$: $\nu_p(a^n - b^n) = \nu_p(a - b) + \nu_p(n)$
- Odd Prime ($p \geq 3$), n odd, $p \mid (a + b)$: $\nu_p(a^n + b^n) = \nu_p(a + b) + \nu_p(n)$
- Even Prime ($p = 2$), a, b odd: $\nu_2(a^n - b^n) = \nu_2(a - b) + \nu_2(a + b) + \nu_2(n) - 1$ (for even n)

3.18 Fibonacci Properties & Fast Doubling

Computes $\{F_n, F_{n+1}\}$ in $O(\log n)$ using $F_{2k} = F_k(2F_{k+1} - F_k), F_{2k+1} = F_k^2 + F_{k+1}^2$.

```
pair<int, int> fib_fast(int n) {
    if (n == 0) return {0, 1};
    auto [a, b] = fib_fast(n >> 1);
    int c = (a * (2 * b - a + m) % M) % M;
    int d = (a * a + b * b) % M;
    if (n & 1) return {d, (c + d) % M};
    return {c, d};
}
```

```
}
```

4. Combinatorics & Discrete Math

4.1 Permutations, Combinations & Modular nCr

Precomputes factorials and inverse factorials up to N in $O(N)$ for $O(1)$ queries.

```
vector<int> fact(N + 5), ifact(N + 5);
void precompute_nCr() {
    fact[0] = 1;
    for (int i = 1; i ≤ N; i++) fact[i] = fact[i - 1] * i % M;
    ifact[N] = binexp(fact[N], M - 2, M);
    for (int i = N - 1; i ≥ 0; i--) ifact[i] = ifact[i + 1] * (i + 1) % M;
}
int nCr(int n, int r) {
    if (r < 0 || r > n) return 0;
    return fact[n] * ifact[r] % M * ifact[n - r] % M;
}
int nPr(int n, int r) {
    if (r < 0 || r > n) return 0;
    return fact[n] * ifact[n - r] % M;
}
```

4.2 Binomial Coefficient Properties

Core identities for manipulating combinations in CP math problems.

- Symmetry: $\binom{n}{k} = \binom{n}{n-k}$
- Pascal's Rule: $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$
- Absorption (Committee): $k \binom{n}{k} = n \binom{n-1}{k-1}$
- Total Subsets: $\sum_{k=0}^n \binom{n}{k} = 2^n$
- Alternating Sum: $\sum_{k=0}^n (-1)^k \binom{n}{k} = 0$
- Even/Odd Sums: $\sum_{k \text{ even}} \binom{n}{k} = \sum_{k \text{ odd}} \binom{n}{k} = 2^{n-1}$
- Vandermonde's Convolution: $\sum_{k=0}^r \binom{m}{k} \binom{n}{r-k} = \binom{m+n}{r}$
- Sum of Squares: $\sum_{k=0}^n \binom{n}{k}^2 = \binom{2n}{n}$
- Weighted Sum: $\sum_{k=1}^n k \binom{n}{k} = n \cdot 2^{n-1}$

4.3 Hockey-Stick Identity (Pyramid Formula for nCr)

Summing along a diagonal or column in Pascal's triangle.

- Column Sum: $\sum_{i=r}^n \binom{i}{r} = \binom{n+1}{r+1}$
- Diagonal Sum: $\sum_{i=0}^k \binom{n+i}{i} = \binom{n+k+1}{k}$
- Pascal's Pyramid (3D / Multinomial): $\binom{n}{i,j,k} = \frac{n!}{i!j!k!}$ ($i + j + k = n$) Each entry is the sum of the 3 adjacent entries in the layer above: $\binom{n}{i,j,k} = \binom{n-1}{i-1,j,k} + \binom{n-1}{i,j-1,k} + \binom{n-1}{i,j,k-1}$

4.4 Stars and Bars (Ballot Problems)

Distributing n identical objects into k distinct containers.

- Non-negative integers ($x_i \geq 0$): $\binom{n+k-1}{k-1}$
- Strictly positive integers ($x_i \geq 1$): $\binom{n-1}{k-1}$
- Lower bounded ($x_i \geq c_i$): Let $n' = n - \sum c_i$; ways $= \binom{n'+k-1}{k-1}$

4.5 Catalan Numbers (Formulas & CP Explanations)

Sequence: 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, ...

- Explicit Formula: $C_n = \frac{1}{n+1} \binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1} = \frac{(2n)!}{(n+1)!n!}$
- Recurrence: $C_0 = 1, \quad C_{n+1} = \sum_{i=0}^n C_i C_{n-i} = \frac{2(2n+1)}{n+2} C_n$
- CP Interpretation & Applications:

- Balanced Parentheses: Number of valid expressions with n pairs of '(' and ')
- Dyck Paths: Monotonic grid walks from $(0, 0)$ to (n, n) that never cross strictly above the diagonal $y = x$.
- Polygon Triangulation: Ways to divide a convex polygon of $n + 2$ vertices into n triangles with non-intersecting diagonals.
- Full Binary Trees: Number of distinct rooted binary trees with $n + 1$ leaves (n internal nodes).
- Non-intersecting Chords: Ways to pair $2n$ points on a circle with n non-intersecting chords.
- Stack Permutations: Permutations of length n that can be sorted using a single stack (avoiding pattern 231).

```
int catalan(int n) {
    return nCr(2 * n, n) * modInvPrime(n + 1) % M;
}
```

4.6 Derangements $D(n)$

Permutations where no element appears in its original position ($\pi(i) \neq i$).

- Sequence: 0, 1, 2, 9, 44, 265, 1854, 14833, ...
- Recurrence: $D(n) = (n - 1)(D(n - 1) + D(n - 2)) = nD(n - 1) + (-1)^n$
- Closed form: $D(n) = n! \sum_{k=0}^n \frac{(-1)^k}{k!} \approx \lfloor \frac{n!}{e} \rfloor$

4.7 Stirling Numbers of 1st and 2nd Kind

Cycle arrangements and set partitions.

- 1st Kind $\begin{bmatrix} n \\ k \end{bmatrix}$: Permutations of n elements with exactly k disjoint cycles. $\begin{bmatrix} n \\ k \end{bmatrix} = \begin{bmatrix} n-1 \\ k-1 \end{bmatrix} + (n-1) \begin{bmatrix} n-1 \\ k \end{bmatrix}, \quad \sum_{k=0}^n \begin{bmatrix} n \\ k \end{bmatrix} = n!$
- 2nd Kind $\left\{ \begin{matrix} n \\ k \end{matrix} \right\}$: Ways to partition n distinct elements into k non-empty sets. $\left\{ \begin{matrix} n \\ k \end{matrix} \right\} = \left\{ \begin{matrix} n-1 \\ k-1 \end{matrix} \right\} + k \left\{ \begin{matrix} n-1 \\ k \end{matrix} \right\}, \quad \left\{ \begin{matrix} n \\ k \end{matrix} \right\} = \frac{1}{k!} \sum_{i=0}^k (-1)^{k-i} \binom{k}{i} i^n$
- Bell Numbers $B(n) = \sum_{k=0}^n \left\{ \begin{matrix} n \\ k \end{matrix} \right\}$: Total ways to partition n distinct elements.

4.8 Lucas' Theorem

Computes $\binom{n}{r} \pmod p$ for large n, r when modulo p is a small prime.

```
int lucas(int n, int r, int p) {
    if (r == 0) return 1;
    int ni = n % p, ri = r % p;
    if (ni < ri) return 0;
    return lucas(n / p, r / p, p) * nCr(ni, ri) % p;
}
```

4.9 Inclusion-Exclusion Principle

Computes size of union of properties $|\bigcup A_i| = \sum_{k=1}^n (-1)^{k-1} \sum_{|J|=k} |\bigcap_{j \in J} A_j|$.

```
// Count elements satisfying none of n conditions
template<typename Func>
int inclusion_exclusion(int n, Func f) {
    int ans = 0;
    for (int mask = 0; mask < (1 << n); mask++) {
        int sign = (__builtin_popcount(mask) & 1) ? -1 : 1;
        ans += sign * f(mask);
    }
    return ans;
}
```

4.10 Burnside's Lemma (Colorings Under Symmetry)

Number of orbits under group actions: $|X/G| = \frac{1}{|G|} \sum_{g \in G} |X^g|$ where $|X^g| = k^{c(g)}$.

- Necklaces with n beads and k colors under rotation: $\frac{1}{n} \sum_{i=0}^{n-1} k^{\gcd(i,n)}$
- Bracelets (rotation + reflection): Add reflection symmetries to group size.

5. Data Structures

5.1 Policy Based Data Structures (PBDS Ordered Set)

Supports `find_by_order(k)` and `order_of_key(x)` in $O(\log N)$. Multiset uses `pair<T, int>` to avoid UB.

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
```

```
template <class T, class Cmp = less<T>>
using ordered_set = tree<T, null_type, Cmp, rb_tree_tag,
    tree_order_statistics_node_update>;
```

```
// Safe Multiset without UB:
ordered_set<pair<int, int>> ost;
int timer = 0;
void insert_val(int x) { ost.insert({x, ++timer}); }
void erase_val(int x) {
    auto it = ost.lower_bound({x, 0});
    if (it != ost.end() && it->first == x) ost.erase(it);
}
int order_of(int x) { return ost.order_of_key({x, 0}); } // count < x
int kth_element(int k) { return ost.find_by_order(k) ->first; } // 0-indexed
```

5.2 Disjoint Set Union (DSU)

Standard DSU with path compression and union by size. Tracks component count and sizes.

```
struct DSU {
    vector<int> p, sz;
    int comps;
    DSU(int n) : p(n + 1), sz(n + 1, 1), comps(n) {
        iota(p.begin(), p.end(), 0);
    }
    int find(int x) {
```

```
        return p[x] == x ? x : p[x] = find(p[x]);
    }
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        p[b] = a;
        sz[a] += sz[b];
        comps--;
        return true;
    }
    int size(int x) { return sz[find(x)]; }
};
```

5.3 Rollback DSU (Undoable Disjoint Sets)

No path compression; union by rank only. Used for offline dynamic connectivity and divide-and-conquer.

```
struct RollbackDSU {
    vector<int> p, sz;
    vector<pair<int, int>> history;
    RollbackDSU(int n) : p(n + 1), sz(n + 1, 1) {
        iota(p.begin(), p.end(), 0);
    }
    int find(int x) {
        while (x != p[x]) x = p[x];
        return x;
    }
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        history.push_back({b, sz[a]});
        p[b] = a;
        sz[a] += sz[b];
        return true;
    }
    int time() { return history.size(); }
    void rollback(int t) {
        while ((int)history.size() > t) {
            auto [b, old_sz_a] = history.back(); history.pop_back();
            int a = p[b];
            sz[a] = old_sz_a;
            p[b] = b;
        }
    }
};
```

5.4 Fenwick Tree (Binary Indexed Tree)

1-indexed point update, range query, and $O(\log N)$ binary lifting search for prefix sums.

```
struct Fenwick {
    int n;
    vector<int> tree;
    Fenwick(int n) : n(n), tree(n + 1, 0) {}
    void add(int i, int delta) {
        for (; i <= n; i += i & -i) tree[i] += delta;
    }
    int query(int i) {
        int sum = 0;
        for (; i > 0; i -= i & -i) sum += tree[i];
        return sum;
    }
    int query(int l, int r) { return query(r) - query(l - 1); }
    int find_kth(int k) { // Find smallest index with prefix sum >= k
        int idx = 0;
        for (int j = 1 << 20; j > 0; j >= 1) {
            if (idx + j <= n && tree[idx + j] < k) {
                idx += j;
                k -= tree[idx];
            }
        }
    }
};
```

```
        return idx + 1;
    }
};

5.5 Range Update & Range Query Fenwick (2-BIT Trick)
Supports range add and range sum using  $p(i) = i \cdot \text{bit}_1(i) - \text{bit}_2(i)$ .
struct RangeFenwick {
    int n;
    vector<int> b1, b2;
    RangeFenwick(int n) : n(n), b1(n + 2, 0), b2(n + 2, 0) {}
    void add_internal(vector<int>& b, int i, int v) {
        for (; i <= n; i += i & -i) b[i] += v;
    }
    void range_add(int l, int r, int v) {
        add_internal(b1, l, v); add_internal(b1, r + 1, -v);
        add_internal(b2, l, v * (1 - 1)); add_internal(b2, r + 1, -v * r
        );
    }
    int prefix_sum(int i) {
        int s1 = 0, s2 = 0;
        for (int x = i; x > 0; x -= x & -x) s1 += b1[x], s2 += b2[x];
        return s1 * i - s2;
    }
    int range_sum(int l, int r) { return prefix_sum(r) - prefix_sum(l
        - 1); }
};
```

5.6 2D Fenwick Tree

Point update and 2D prefix/submatrix sum in $O(\log N \log M)$.

```
struct Fenwick2D {
    int n, m;
    vector<vector<int>> tree;
    Fenwick2D(int n, int m) : n(n), m(m), tree(n + 1, vector<int>(m +
        1, 0)) {}
    void add(int x, int y, int v) {
        for (int i = x; i <= n; i += i & -i)
            for (int j = y; j <= m; j += j & -j) tree[i][j] += v;
    }
    int query(int x, int y) {
        int sum = 0;
        for (int i = x; i > 0; i -= i & -i)
            for (int j = y; j > 0; j -= j & -j) sum += tree[i][j];
        return sum;
    }
    int query(int x1, int y1, int x2, int y2) {
        return query(x2, y2) - query(x1 - 1, y2) - query(x2, y1 - 1) +
            query(x1 - 1, y1 - 1);
    }
};
```

5.7 Segment Tree (User's Point Update, Range Sum)

Zero-indexed array implementation matching user snippet.

```
class SGTree {
public:
    vector<int> seg;
    SGTree(int n) { seg.resize(4 * n + 1, 0); }
    void build(int idx, int l, int r, vector<int> &v) {
        if (l == r) { seg[idx] = v[l]; return; }
        int mid = (l + r) / 2;
        build(2 * idx + 1, l, mid, v);
        build(2 * idx + 2, mid + 1, r, v);
        seg[idx] = seg[2 * idx + 1] + seg[2 * idx + 2];
    }
    void update(int idx, int l, int r, int pos, int val) {
        if (l == r) { seg[idx] = val; return; }
        int mid = (l + r) / 2;
        if (pos <= mid) update(2 * idx + 1, l, mid, pos, val);
        else update(2 * idx + 2, mid + 1, r, pos, val);
        seg[idx] = seg[2 * idx + 1] + seg[2 * idx + 2];
    }
    int query(int idx, int l, int r, int ql, int qr) {
```

```

    if (qr < l || r < ql) return 0;
    if (ql ≤ l && r ≤ qr) return seg[idx];
    int mid = (l + r) / 2;
    return query(2 * idx + 1, l, mid, ql, qr) + query(2 * idx + 2,
        mid + 1, r, ql, qr);
}
};

```

5.8 Lazy Segment Tree (Range Add & Range Sum/Min/Max)

General range update and range query in $O(\log N)$.

```

struct LazySeg {
    int n;
    vector<int> tree, lazy;
    LazySeg(int n) : n(n), tree(4 * n + 1, 0), lazy(4 * n + 1, 0) {}
    void push(int node, int l, int r) {
        if (!lazy[node]) return;
        tree[node] += (r - l + 1) * lazy[node];
        if (l ≠ r) {
            lazy[node * 2] += lazy[node];
            lazy[node * 2 + 1] += lazy[node];
        }
        lazy[node] = 0;
    }
    void update(int node, int l, int r, int ql, int qr, int val) {
        push(node, l, r);
        if (qr < l || r < ql) return;
        if (ql ≤ l && r ≤ qr) {
            lazy[node] += val;
            push(node, l, r);
            return;
        }
        int mid = (l + r) / 2;
        update(node * 2, l, mid, ql, qr, val);
        update(node * 2 + 1, mid + 1, r, ql, qr, val);
        tree[node] = tree[node * 2] + tree[node * 2 + 1];
    }
    int query(int node, int l, int r, int ql, int qr) {
        push(node, l, r);
        if (qr < l || r < ql) return 0;
        if (ql ≤ l && r ≤ qr) return tree[node];
        int mid = (l + r) / 2;
        return query(node * 2, l, mid, ql, qr) + query(node * 2 + 1, mid
            + 1, r, ql, qr);
    }
};

```

5.9 Persistent Segment Tree (K-th Smallest in Range)

Creates new nodes on update. Queries k-th element in range $[l, r]$ in $O(\log N)$.

```

struct PST {
    static const int MAXN = 200005, LG = 20;
    int lc[MAXN * LG], rc[MAXN * LG], sum[MAXN * LG], roots[MAXN], sz
        = 1;
    int update(int prev, int l, int r, int pos, int val) {
        int cur = sz++;
        lc[cur] = lc[prev]; rc[cur] = rc[prev]; sum[cur] = sum[prev] +
            val;
        if (l == r) return cur;
        int mid = (l + r) / 2;
        if (pos ≤ mid) lc[cur] = update(lc[prev], l, mid, pos, val);
        else rc[cur] = update(rc[prev], mid + 1, r, pos, val);
        return cur;
    }
    int query_kth(int u, int v, int l, int r, int k) {
        if (l == r) return l;
        int count = sum[lc[v]] - sum[lc[u]];
        int mid = (l + r) / 2;
        if (k ≤ count) return query_kth(lc[u], lc[v], l, mid, k);
        return query_kth(rc[u], rc[v], mid + 1, r, k - count);
    }
};

```

5.10 Sparse Table (1D RMQ in $O(1)$)

Idempotent range minimum / maximum / gcd queries. Precomputation $O(N \log N)$.

```

struct SparseTable {
    int n, K;
    vector<vector<int>> st;
    SparseTable(const vector<int>& a) : n(a.size()), K(__lg(n) + 1) {
        st.assign(K, vector<int>(n));
        st[0] = a;
        for (int j = 1; j < K; j++)
            for (int i = 0; i + (1 << j) ≤ n; i++)
                st[j][i] = min(st[j - 1][i], st[j - 1][i + (1 << (j - 1))]);
    }
    int query(int l, int r) {
        int j = __lg(r - l + 1);
        return min(st[j][l], st[j][r - (1 << j) + 1]);
    }
};

```

5.11 Disjoint Sparse Table (Non-Idempotent $O(1)$)

Answers arbitrary associative range queries (sum, XOR, matrix mul) in $O(1)$.

```

struct DisjointSparseTable {
    int n, levels;
    vector<vector<int>> table;
    DisjointSparseTable(vector<int> a) {
        n = a.size();
        int m = 1;
        while (m < n) m <= 1;
        a.resize(m, 0);
        levels = __lg(m) + 1;
        table.assign(levels, vector<int>(m));
        table[0] = a;
        for (int h = 1; h < levels; h++) {
            int half = 1 << (h - 1);
            for (int i = 0; i < m; i += half * 2) {
                int mid = i + half;
                table[h][mid - 1] = a[mid - 1];
                for (int j = mid - 2; j ≥ i; j--) table[h][j] = table[h][j
                    + 1] + a[j];
                table[h][mid] = a[mid];
                for (int j = mid + 1; j < i + half * 2; j++) table[h][j] =
                    table[h][j - 1] + a[j];
            }
        }
    }
    int query(int l, int r) {
        if (l == r) return table[0][l];
        int h = __lg(1 ^ r) + 1;
        return table[h][l] + table[h][r];
    }
};

```

5.12 Alphabet Trie & Binary Trie (Max XOR)

Prefix trees for strings and numbers. Max XOR pair query in $O(B)$ time.

```

// String Trie:
struct TrieNode {
    TrieNode* nxt[26] = {};
    bool is_end = false;
};
void insert_str(TrieNode* root, const string &s) {
    TrieNode* cur = root;
    for (char c : s) {
        int x = c - 'a';
        if (!cur->nxt[x]) cur->nxt[x] = new TrieNode();
        cur = cur->nxt[x];
    }
    cur->is_end = true;
}

```

// Binary Trie for Max XOR:

```

struct BitTrie {
    struct Node { int nxt[2] = {-1, -1}; int cnt = 0; };
    vector<Node> tree;
    BitTrie() { tree.push_back({}); }
    void insert(int x) {
        int cur = 0; tree[cur].cnt++;
        for (int i = 30; i ≥ 0; i--) {
            int b = (x >> i) & 1;
            if (tree[cur].nxt[b] == -1) {
                tree[cur].nxt[b] = tree.size();
                tree.push_back({});
            }
            cur = tree[cur].nxt[b];
            tree[cur].cnt++;
        }
    }
    int max_xor(int x) {
        int cur = 0, res = 0;
        for (int i = 30; i ≥ 0; i--) {
            int b = (x >> i) & 1;
            if (tree[cur].nxt[b ^ 1] ≠ -1 && tree[tree[cur].nxt[b ^ 1]].
                cnt > 0) {
                res |= (1 << i);
                cur = tree[cur].nxt[b ^ 1];
            } else {
                cur = tree[cur].nxt[b];
            }
        }
        return res;
    }
};

```

5.13 Monotonic Stack & Monotonic Queue

Nearest Smaller to Left in $O(N)$; Sliding window maximum in $O(1)$ amortized.

```

// Nearest Smaller to Left (Values):
vector<int> nearest_smaller_left(const vector<int>& a) {
    int n = a.size();
    vector<int> ans(n, -1);
    stack<int> st;
    for (int i = 0; i < n; i++) {
        while (!st.empty() && st.top() ≥ a[i]) st.pop();
        if (!st.empty()) ans[i] = st.top();
        st.push(a[i]);
    }
    return ans;
}

```

```

// Sliding Window Maximum (Window size K):
vector<int> sliding_max(const vector<int>& a, int k) {
    deque<int> dq;
    vector<int> res;
    for (int i = 0; i < (int)a.size(); i++) {
        while (!dq.empty() && dq.front() ≤ i - k) dq.pop_front();
        while (!dq.empty() && a[dq.back()] ≤ a[i]) dq.pop_back();
        dq.push_back(i);
        if (i ≥ k - 1) res.push_back(a[dq.front()]);
    }
    return res;
}

```

5.14 Mo's Algorithm (Snake Order)

Offline range queries in $O((N + Q)\sqrt{N})$. Alternating sorting cuts pointer moves.

```

struct Query {
    int l, r, id;
};
void solve_mo(int n, vector<Query>& queries) {
    int B = max(1LL, (int)sqrt(n));
    sort(queries.begin(), queries.end(), [&](const Query& a, const
        Query& b) {
        if (a.l / B ≠ b.l / B) return a.l / B < b.l / B;
    });
}

```

```

    return (a.l / B) & 1 ? a.r < b.r : a.r > b.r;
});
int curL = 0, curR = -1;
auto add = [&](int idx) { /* include a[idx] */ };
auto remove = [&](int idx) { /* remove a[idx] */ };
for (auto &q : queries) {
    while (curL > q.l) add(--curL);
    while (curR < q.r) add(++curR);
    while (curL < q.l) remove(curL++);
    while (curR > q.r) remove(curR--);
    // ans[q.id] = current_answer;
}
}

```

5.15 Small-to-Large Merging

When merging data sets between components, always merge the smaller into the larger for $O(N \log N)$ total time.

```

void merge_sets(vector<int>& comp_a, vector<int>& comp_b) {
    if (comp_a.size() < comp_b.size()) swap(comp_a, comp_b);
    for (int val : comp_b) comp_a.push_back(val);
    comp_b.clear();
}

```

5.16 Coordinate Compression

Maps large coordinate values to $[0, K - 1]$ while preserving order.

```

vector<int> vals = a;
sort(vals.begin(), vals.end());
vals.erase(unique(vals.begin(), vals.end()), vals.end());
for (int &x : a) x = lower_bound(vals.begin(), vals.end(), x) - vals
    .begin();

```

6. String Algorithms

6.1 Double Polynomial Rolling Hash

Static $O(1)$ substring hash queries with two moduli to eliminate collision risks.

```

struct StringHash {
    int n;
    const int mod1 = 1000000007, mod2 = 1000000009;
    const int base1 = 313, base2 = 317;
    vector<int> h1, h2, p1, p2;
    StringHash(const string& s) : n(s.size()), h1(n + 1, 0), h2(n + 1,
        0), p1(n + 1, 1), p2(n + 1, 1) {
        for (int i = 0; i < n; i++) {
            p1[i + 1] = ( (__int128)p1[i] * base1 % mod1;
            p2[i + 1] = ( (__int128)p2[i] * base2 % mod2;
            h1[i + 1] = (( (__int128)h1[i] * base1 + s[i]) % mod1;
            h2[i + 1] = (( (__int128)h2[i] * base2 + s[i]) % mod2;
        }
    }
    pair<int, int> get_hash(int l, int r) { // 0-indexed [l, r]
        int x1 = (h1[r + 1] - ( (__int128)h1[l] * p1[r - l + 1]) % mod1;
        if (x1 < 0) x1 += mod1;
        int x2 = (h2[r + 1] - ( (__int128)h2[l] * p2[r - l + 1]) % mod2;
        if (x2 < 0) x2 += mod2;
        return {x1, x2};
    }
};

```

6.2 Dynamic String Hash (Point Update Segment Tree)

Supports single-character replacement and $O(\log N)$ substring hash queries.

```

struct DynamicHash {
    int n; string s;
    vector<int> h, p;
    const int base = 313, mod = 1000000007;
    DynamicHash(string str) : n(str.size()), s(str), h(4 * n + 1), p(n
        + 1, 1) {
        for (int i = 1; i ≤ n; i++) p[i] = (int)(( (__int128)p[i - 1] *
            base % mod);
        build(1, 0, n - 1);
    }
}

```

```

void build(int node, int l, int r) {
    if (l == r) { h[node] = s[l]; return; }
    int mid = (l + r) / 2;
    build(node * 2, l, mid);
    build(node * 2 + 1, mid + 1, r);
    h[node] = (( (__int128)h[node * 2] * p[r - mid] + h[node * 2 + 1])
        % mod;
}
void update(int node, int l, int r, int pos, char c) {
    if (l == r) { h[node] = s[pos] = c; return; }
    int mid = (l + r) / 2;
    if (pos ≤ mid) update(node * 2, l, mid, pos, c);
    else update(node * 2 + 1, mid + 1, r, pos, c);
    h[node] = (( (__int128)h[node * 2] * p[r - mid] + h[node * 2 + 1])
        % mod;
}
int query(int node, int l, int r, int ql, int qr) {
    if (ql ≤ l && r ≤ qr) return h[node];
    int mid = (l + r) / 2;
    if (qr ≤ mid) return query(node * 2, l, mid, ql, qr);
    if (ql > mid) return query(node * 2 + 1, mid + 1, r, ql, qr);
    int left_h = query(node * 2, l, mid, ql, qr);
    int right_h = query(node * 2 + 1, mid + 1, r, ql, qr);
    return (( (__int128)left_h * p[min(r, qr) - mid] + right_h) % mod;
}
};

```

6.3 Prefix Function (KMP) & Pattern Matching

$\pi[i]$ is the length of the longest proper prefix of $s[0..i]$ that is also a suffix of $s[0..i]$.

```

vector<int> prefix_function(const string& s) {
    int n = s.size();
    vector<int> pi(n, 0);
    for (int i = 1; i < n; i++) {
        int j = pi[i - 1];
        while (j > 0 && s[i] ≠ s[j]) j = pi[j - 1];
        if (s[i] == s[j]) j++;
        pi[i] = j;
    }
    return pi;
}
vector<int> kmp_search(const string& text, const string& pat) {
    string cur = pat + "#" + text;
    vector<int> pi = prefix_function(cur), matches;
    for (int i = pat.size() + 1; i < (int)cur.size(); i++) {
        if (pi[i] == (int)pat.size()) matches.push_back(i - 2 * pat.size
            ());
    }
    return matches;
}

```

6.4 Z-Algorithm

$Z[i]$ is the length of the longest common prefix of s and $s[i..n - 1]$ in $O(N)$ time.

```

vector<int> z_function(const string& s) {
    int n = s.size();
    vector<int> z(n, 0);
    for (int i = 1, l = 0, r = 0; i < n; i++) {
        if (i ≤ r) z[i] = min(r - i + 1, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
        if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
    }
    return z;
}

```

6.5 Manacher's Algorithm (Palindromes in $O(N)$)

Computes all palindrome radii in $O(N)$. Interleaving with '#' handles odd and even palindromes uniformly.

- Simple Explanation: Transform string "aba" to

"@#a#b#a#\$". Then every palindrome (odd or even) has an odd length with a single center.

- Radius Reuse: When expanding around center i , if $i < r$, we copy the previously computed radius from its symmetric mirror $i' = l + r - i$: $p[i] = \min(r - i, p[i'])$. We only expand character-by-character if the palindrome extends beyond the current known rightmost boundary r . Since r strictly increases, the overall time is $O(N)$.

```

// Returns array p where p[i] is radius in transformed string
vector<int> manacher(const string& s) {
    string t = "@";
    for (char c : s) { t += "#"; t += c; }
    t += "$";
    int n = t.size(), l = 0, r = 0;
    vector<int> p(n, 0);
    for (int i = 1; i < n - 1; i++) {
        int mirror = l + r - i;
        if (i < r) p[i] = min(r - i, p[mirror]);
        while (t[i + p[i] + 1] == t[i - p[i] - 1]) p[i]++;
        if (i + p[i] > r) { l = i - p[i]; r = i + p[i]; }
    }
    return p; // Longest palindrome substring length is max_element(p)
}
bool is_palindrome(const vector<int>& p, int orig_l, int orig_r) {
    int center = orig_l + orig_r + 2; // In 0-indexed original string
    int len = orig_r - orig_l + 1;
    return p[center] - 1 ≥ len;
}

```

6.6 Suffix Array & LCP ($O(N \log N)$)

Constructs suffix array using prefix doubling. Kasai's algorithm finds LCP array in $O(N)$.

```

struct SuffixArray {
    int n;
    vector<int> sa, lcp, rank;
    SuffixArray(string s) : n(s.size() + 1), sa(n), lcp(n), rank(n) {
        s += "$";
        vector<int> k(n);
        for (int i = 0; i < n; i++) { sa[i] = i; rank[i] = s[i]; }
        for (int len = 1; len < n; len ≤ 1) {
            auto cmp = [&](int i, int j) {
                if (rank[i] ≠ rank[j]) return rank[i] < rank[j];
                int ri = (i + len < n ? rank[i + len] : -1);
                int rj = (j + len < n ? rank[j + len] : -1);
                return ri < rj;
            };
            sort(sa.begin(), sa.end(), cmp);
            k[sa[0]] = 0;
            for (int i = 1; i < n; i++) k[sa[i]] = k[sa[i - 1]] + cmp(sa[i
                - 1], sa[i]);
            rank = k;
            if (rank[sa[n - 1]] == n - 1) break;
        }
        // Kasai's algorithm for LCP:
        for (int i = 0, h = 0; i < n - 1; i++) {
            int pi = rank[i], j = sa[pi - 1];
            while (s[i + h] == s[j + h]) h++;
            lcp[pi] = h;
            if (h > 0) h--;
        }
    }
    // Total distinct substrings: n*(n-1)/2 - sum(lcp[i])
};

```

6.7 Aho-Corasick Automaton

Multi-pattern string matching in $O(\text{Text} + \sum |\text{Patterns}|)$. Builds BFS failure links.

```
struct AhoCorasick {
    struct Node { int nxt[26] = {}, fail = 0, match_cnt = 0; };
    vector<Node> t;
    AhoCorasick() { t.push_back({}); }
    void insert(const string& s) {
        int cur = 0;
        for (char c : s) {
            int x = c - 'a';
            if (!t[cur].nxt[x]) { t[cur].nxt[x] = t.size(); t.push_back({}); }
            cur = t[cur].nxt[x];
        }
        t[cur].match_cnt++;
    }
    void build() {
        queue<int> q;
        for (int c = 0; c < 26; c++) if (t[0].nxt[c]) q.push(t[0].nxt[c]);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int c = 0; c < 26; c++) {
                if (t[u].nxt[c]) {
                    t[t[u].nxt[c]].fail = t[t[u].fail].nxt[c];
                    t[t[u].nxt[c]].match_cnt += t[t[t[u].nxt[c]].fail].match_cnt;
                    q.push(t[u].nxt[c]);
                } else {
                    t[u].nxt[c] = t[t[u].fail].nxt[c];
                }
            }
        }
    }
};
```

6.8 Booth’s Algorithm (Minimum Lexicographical Rotation)

Finds starting index of lexicographically smallest cyclic shift of s in $O(N)$ time.

```
int min_cyclic_string(string s) {
    s += s;
    int n = s.size(), i = 0, ans = 0;
    while (i < n / 2) {
        ans = i;
        int j = i + 1, k = i;
        while (j < n && s[k] <= s[j]) {
            if (s[k] < s[j]) k = i;
            else k++;
            j++;
        }
        while (i <= k) i += j - k;
    }
    return ans;
}
```

6.9 String Utility Helpers

Fast conversions, tokenization, and character properties.

```
// Tokenize by whitespace:
vector<string> split(const string& s) {
    stringstream ss(s); string w;
    vector<string> res;
    while (ss >> w) res.push_back(w);
    return res;
}
// String <=> Number: stoi(s), stoll(s), stod(s), to_string(x)
// Char tests: islower(c), isupper(c), isdigit(c), isalpha(c),
//              tolower(c), toupper(c)
```

7. Graph Theory

7.1 Graph Traversals (DFS, BFS, 0-1 BFS, Multi-Source)

Core search algorithms for unweighted and 0/1 weighted graphs.

```
// 0-1 BFS in  $O(V + E)$  using std::deque:
vector<int> bfs01(int src, int n, const vector<vector<pair<int, int>>& g) {
    vector<int> dist(n + 1, INF);
    deque<int> dq;
    dist[src] = 0; dq.push_front(src);
    while (!dq.empty()) {
        int u = dq.front(); dq.pop_front();
        for (auto [v, w] : g[u]) {
            if (dist[v] > dist[u] + w) {
                dist[v] = dist[u] + w;
                if (w == 0) dq.push_front(v);
                else dq.push_back(v);
            }
        }
    }
    return dist;
}
```

7.2 Dijkstra’s Algorithm (Single Source Shortest Path)

Finds shortest paths with non-negative edge weights in $O((V + E)\log V)$.

```
vector<int> dijkstra(int s, int n, const vector<vector<pair<int, int>>& g) {
    vector<int> dist(n + 1, INF);
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> pq;
    dist[s] = 0; pq.push({0, s});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue;
        for (auto [v, w] : g[u]) {
            if (dist[v] > dist[u] + w) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
    return dist;
}
```

7.3 Bellman-Ford & Negative Cycle Detection

Finds shortest paths with negative edges in $O(V \cdot E)$ and identifies negative cycles.

```
struct Edge { int u, v, w; };
vector<int> bellman_ford(int s, int n, const vector<Edge>& edges,
    bool &has_neg_cycle) {
    vector<int> dist(n + 1, INF);
    dist[s] = 0;
    for (int i = 1; i <= n - 1; i++) {
        for (const auto& e : edges) {
            if (dist[e.u] < INF && dist[e.v] > dist[e.u] + e.w) dist[e.v] = dist[e.u] + e.w;
        }
    }
    has_neg_cycle = false;
    for (const auto& e : edges) {
        if (dist[e.u] < INF && dist[e.v] > dist[e.u] + e.w) {
            has_neg_cycle = true;
            dist[e.v] = -INF; // Reachable from negative cycle
        }
    }
    return dist;
}
```

7.4 Floyd-Warshall (All-Pairs Shortest Path)

Computes all-pairs shortest paths in $O(V^3)$. Propagates negative cycles.

```
void floyd_warshall(int n, vector<vector<int>>& d) {
    for (int i = 1; i <= n; i++) d[i][i] = min(d[i][i], 0LL);
    for (int k = 1; k <= n; k++)
```

```
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            if (d[i][k] < INF && d[k][j] < INF)
                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
    // Propagate -INF for negative cycles:
    for (int k = 1; k <= n; k++)
        if (d[k][k] < 0)
            for (int i = 1; i <= n; i++)
                for (int j = 1; j <= n; j++)
                    if (d[i][k] < INF && d[k][j] < INF) d[i][j] = -INF;
}
```

7.5 Strongly Connected Components (Kosaraju’s)

Two DFS passes: 1st on G for finish times; 2nd on G^T to extract SCCs. Builds condensation DAG.

```
struct KosarajuSCC {
    int n, comps = 0;
    vector<vector<int>> g, rg, dag;
    vector<int> order, comp;
    vector<bool> vis;
    KosarajuSCC(int n) : n(n), g(n + 1), rg(n + 1), comp(n + 1, -1),
        vis(n + 1, false) {}
    void add_edge(int u, int v) { g[u].push_back(v); rg[v].push_back(u); }
    void dfs1(int u) {
        vis[u] = true;
        for (int v : g[u]) if (!vis[v]) dfs1(v);
        order.push_back(u);
    }
    void dfs2(int u, int c) {
        comp[u] = c;
        for (int v : rg[u]) if (comp[v] == -1) dfs2(v, c);
    }
    void build() {
        for (int i = 1; i <= n; i++) if (!vis[i]) dfs1(i);
        reverse(order.begin(), order.end());
        for (int u : order) if (comp[u] == -1) dfs2(u, comps++);
        dag.resize(comps);
        for (int u = 1; u <= n; u++) {
            for (int v : g[u]) if (comp[u] != comp[v]) dag[comp[u]].push_back(comp[v]);
        }
    }
};
```

7.6 Strongly Connected Components (Tarjan’s)

Single-pass DFS with explicit stack and entry/low values to find SCCs in $O(V + E)$.

```
struct TarjanSCC {
    int n, timer = 0, comps = 0;
    vector<vector<int>> g;
    vector<int> tin, low, comp;
    vector<bool> in_st;
    stack<int> st;
    TarjanSCC(int n) : n(n), g(n + 1), tin(n + 1, -1), low(n + 1, -1),
        comp(n + 1, -1), in_st(n + 1, false) {}
    void add_edge(int u, int v) { g[u].push_back(v); }
    void dfs(int u) {
        tin[u] = low[u] = ++timer;
        st.push(u); in_st[u] = true;
        for (int v : g[u]) {
            if (tin[v] == -1) {
                dfs(v);
                low[u] = min(low[u], low[v]);
            } else if (in_st[v]) {
                low[u] = min(low[u], tin[v]);
            }
        }
        if (low[u] == tin[u]) {
            while (true) {
                int v = st.top(); st.pop();
```

```

    in_st[v] = false;
    comp[v] = comps;
    if (v == u) break;
}
comps++;
}
}
void build() { for (int i = 1; i ≤ n; i++) if (tin[i] == -1) dfs(i); }
};

```

7.7 2-SAT Solver (via SCC)

Solves 2-SAT clause constraints ($u \vee v$). Assigns true/false values in $O(V + E)$.

```

struct TwoSAT {
    int n;
    KosarajuSCC scc;
    TwoSAT(int n) : n(n), scc(2 * n) {}
    // Node x: 2*x (false), 2*x + 1 (true)
    void add_clause(int u, bool val_u, int v, bool val_v) {
        int pu = 2 * u + (val_u ? 1 : 0);
        int pv = 2 * v + (val_v ? 1 : 0);
        scc.add_edge(pu ^ 1, pv); // ~u => v
        scc.add_edge(pv ^ 1, pu); // ~v => u
    }
    bool solve(vector<bool>& assignment) {
        scc.build();
        assignment.resize(n + 1);
        for (int i = 1; i ≤ n; i++) {
            if (scc.comp[2 * i] == scc.comp[2 * i + 1]) return false; // Unsatisfiable
            assignment[i] = scc.comp[2 * i + 1] < scc.comp[2 * i];
        }
        return true;
    }
};

```

7.8 Articulation Points & Bridges

Finds all cut vertices and bridge edges in an undirected graph in $O(V + E)$.

```

struct BridgesAndArt {
    int n, timer = 0;
    vector<vector<int>> g;
    vector<int> tin, low;
    vector<bool> is_art;
    vector<pair<int, int>> bridges;
    BridgesAndArt(int n) : n(n), g(n + 1), tin(n + 1, -1), low(n + 1, -1), is_art(n + 1, false) {}
    void add_edge(int u, int v) { g[u].push_back(v); g[v].push_back(u); }
    void dfs(int u, int p = -1) {
        tin[u] = low[u] = ++timer;
        int children = 0;
        for (int v : g[u]) {
            if (v == p) continue;
            if (tin[v] ≠ -1) {
                low[u] = min(low[u], tin[v]);
            } else {
                dfs(v, u);
                low[u] = min(low[u], low[v]);
                if (low[v] > tin[u]) bridges.push_back({u, v});
                if (low[v] ≥ tin[u] && p ≠ -1) is_art[u] = true;
                children++;
            }
        }
        if (p == -1 && children > 1) is_art[u] = true;
    }
    void build() { for (int i = 1; i ≤ n; i++) if (tin[i] == -1) dfs(i); }
};

```

7.9 Tree Flattening (Euler Tour)

Maps tree subtrees into contiguous 1D array segments $[tin[u], tout[u]]$ for range data structures.

```

vector<int> tin, tout;
int timer = 0;
void euler_tour(int u, int p, const vector<vector<int>>& g) {
    tin[u] = ++timer;
    for (int v : g[u]) if (v ≠ p) euler_tour(v, u, g);
    tout[u] = timer;
}

```

7.10 Binary Lifting & LCA

Precomputation $O(N \log N)$; LCA and k -th ancestor queries in $O(\log N)$.

```

struct LCA {
    int n, LG;
    vector<vector<int>> up;
    vector<int> depth;
    LCA(int n, const vector<vector<int>>& g, int root = 1) : n(n), LG(
        __lg(n) + 1), up(LG, vector<int>(n + 1, 0)), depth(n + 1, 0) {
        auto dfs = [&](auto self, int u, int p, int d) → void {
            up[0][u] = p; depth[u] = d;
            for (int i = 1; i < LG; i++) up[i][u] = up[i - 1][up[i - 1][u]];
            for (int v : g[u]) if (v ≠ p) self(self, v, u, d + 1);
        };
        dfs(dfs, root, 0, 0);
    }
    int get_lca(int u, int v) {
        if (depth[u] < depth[v]) swap(u, v);
        for (int i = LG - 1; i ≥ 0; i--)
            if (depth[u] - (1 << i) ≥ depth[v]) u = up[i][u];
        if (u == v) return u;
        for (int i = LG - 1; i ≥ 0; i--) {
            if (up[i][u] ≠ up[i][v]) { u = up[i][u]; v = up[i][v]; }
        }
        return up[0][u];
    }
};

```

7.11 Heavy-Light Decomposition (HLD)

Decomposes tree into $O(\log N)$ heavy paths. Supports path queries and updates via segment tree.

```

struct HLD {
    int n, cur_pos = 1;
    vector<int> parent, depth, heavy, head, pos, sz;
    vector<vector<int>> g;
    HLD(int n) : n(n), parent(n + 1), depth(n + 1), heavy(n + 1, -1),
        head(n + 1), pos(n + 1), sz(n + 1, 1) {}
    void add_edge(int u, int v) { g[u].push_back(v); g[v].push_back(u); }
    int dfs(int u, int p, int d) {
        parent[u] = p; depth[u] = d; sz[u] = 1;
        int max_c = 0;
        for (int v : g[u]) {
            if (v == p) continue;
            int c_sz = dfs(v, u, d + 1);
            sz[u] += c_sz;
            if (c_sz > max_c) { max_c = c_sz; heavy[u] = v; }
        }
        return sz[u];
    }
    void decompose(int u, int h) {
        head[u] = h; pos[u] = cur_pos++;
        if (heavy[u] ≠ -1) decompose(heavy[u], h);
        for (int v : g[u]) {
            if (v ≠ parent[u] && v ≠ heavy[u]) decompose(v, v);
        }
    }
    void init(int root = 1) { dfs(root, 0, 0); decompose(root, root); }
};

```

7.12 DSU on Tree (Sack / Small-to-Large on Trees)

Solves offline subtree queries in $O(N \log N)$ by keeping the heavy child's contribution.

```

int sz_tree[N], heavy_child[N];
void dfs_sz(int u, int p, const vector<vector<int>>& g) {
    sz_tree[u] = 1; heavy_child[u] = -1; int max_c = 0;
    for (int v : g[u]) if (v ≠ p) {
        dfs_sz(v, u, g);
        sz_tree[u] += sz_tree[v];
        if (sz_tree[v] > max_c) { max_c = sz_tree[v]; heavy_child[u] = v; }
    }
}
void dfs_sack(int u, int p, bool keep, const vector<vector<int>>& g) {
    {
        for (int v : g[u]) if (v ≠ p && v ≠ heavy_child[u]) dfs_sack(v, u, false, g);
        if (heavy_child[u] ≠ -1) dfs_sack(heavy_child[u], u, true, g);
        // Add light children subtrees and node u to DS
        // Answer queries for u
        if (!keep) { /* Remove all elements in subtree u from DS */ }
    }
}

```

7.13 Dinic's Algorithm (Maximum Flow)

Finds maximum flow in $O(V^2 E)$ (or $O(E\sqrt{V})$ for unit networks) using BFS levels and DFS blocking flow.

```

struct Dinic {
    struct Edge { int to, rev; int cap, flow; };
    int n, s, t;
    vector<vector<Edge>> adj;
    vector<int> level, ptr;
    Dinic(int n, int s, int t) : n(n), s(s), t(t), adj(n + 1), level(n + 1), ptr(n + 1) {}
    void add_edge(int u, int v, int cap) {
        adj[u].push_back({v, (int)adj[v].size(), cap, 0});
        adj[v].push_back({u, (int)adj[u].size() - 1, 0, 0});
    }
    bool bfs() {
        fill(level.begin(), level.end(), -1);
        level[s] = 0; queue<int> q; q.push(s);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (const auto& e : adj[u]) {
                if (e.cap - e.flow > 0 && level[e.to] == -1) {
                    level[e.to] = level[u] + 1; q.push(e.to);
                }
            }
        }
        return level[t] ≠ -1;
    }
    int dfs(int u, int pushed) {
        if (pushed == 0 || u == t) return pushed;
        for (int& cid = ptr[u]; cid < (int)adj[u].size(); cid++) {
            auto& e = adj[u][cid];
            if (level[u] + 1 ≠ level[e.to] || e.cap - e.flow == 0) continue;
            int tr = dfs(e.to, min(pushed, e.cap - e.flow));
            if (tr == 0) continue;
            e.flow += tr;
            adj[e.to][e.rev].flow -= tr;
            return tr;
        }
        return 0;
    }
    int max_flow() {
        int flow = 0;
        while (bfs()) {
            fill(ptr.begin(), ptr.end(), 0);
            while (int pushed = dfs(s, INF)) flow += pushed;
        }
        return flow;
    }
};

```

```
    }
};
```

7.14 Maximum Bipartite Matching (Kuhn's)

Simple $O(V \cdot E)$ augmenting path algorithm for bipartite matching.

```
bool dfs_kuhn(int u, const vector<vector<int>>& g, vector<int>& match, vector<bool>& vis) {
    if (vis[u]) return false;
    vis[u] = true;
    for (int v : g[u]) {
        if (match[v] == -1 || dfs_kuhn(match[v], g, match, vis)) {
            match[v] = u; return true;
        }
    }
    return false;
}

int max_bipartite_matching(int n_left, int n_right, const vector<vector<int>>& g) {
    vector<int> match(n_right + 1, -1);
    int res = 0;
    for (int u = 1; u ≤ n_left; u++) {
        vector<bool> vis(n_left + 1, false);
        if (dfs_kuhn(u, g, match, vis)) res++;
    }
    return res;
}
```

7.15 Tree Diameter (2-BFS)

BFS from node 1 gives farthest node u; BFS from u gives farthest node v and diameter.

```
pair<int, int> bfs_farthest(int start, int n, const vector<vector<int>>& g) {
    int u = 1;
    vector<int> dist(n + 1, -1);
    queue<int> q; q.push(start); dist[start] = 0;
    int far_node = start;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        if (dist[u] > dist[far_node]) far_node = u;
        for (int v : g[u]) if (dist[v] == -1) { dist[v] = dist[u] + 1; q.push(v); }
    }
    return {far_node, dist[far_node]};
}

int tree_diameter(int n, const vector<vector<int>>& g) {
    int u = bfs_farthest(1, n, g).first;
    return bfs_farthest(u, n, g).second;
}
```

8. Dynamic Programming

8.1 Classical DP (Knapsack, Coins, LIS, LCS, Edit Dist)

Essential competitive programming dynamic programming templates.

// 0/1 Knapsack (Reverse iteration):

```
for (int i = 0; i < n; i++)
    for (int j = W; j ≥ wt[i]; j--) dp[j] = max(dp[j], dp[j - wt[i]] + val[i]);
```

// Unbounded Knapsack (Forward iteration):

```
for (int i = 0; i < n; i++)
    for (int j = wt[i]; j ≤ W; j++) dp[j] = max(dp[j], dp[j - wt[i]] + val[i]);
```

// LIS in $O(N \log N)$:

```
vector<int> tails;
for (int x : a) {
    auto it = lower_bound(tails.begin(), tails.end(), x);
    if (it == tails.end()) tails.push_back(x);
    else *it = x;
}
int lis_len = tails.size();
```

// LCS in $O(N \cdot M)$:

```
for (int i = 1; i ≤ n; i++)
    for (int j = 1; j ≤ m; j++)
        dp[i][j] = (s[i-1] == t[j-1] ? dp[i-1][j-1] + 1 : max(dp[i-1][j], dp[i][j-1]));
```

// Edit Distance:

```
for (int i = 1; i ≤ n; i++)
    for (int j = 1; j ≤ m; j++) {
        if (s[i-1] == t[j-1]) dp[i][j] = dp[i-1][j-1];
        else dp[i][j] = 1 + min({dp[i-1][j], dp[i][j-1], dp[i-1][j-1]});
    }
```

8.2 Kadane's Algorithm & Max Submatrix Sum

Max subarray sum in $O(N)$; 2D submatrix max sum in $O(N^2M)$ by collapsing columns.

```
int kadane(const vector<int>& a) {
    int cur = 0, ans = -INF;
    for (int x : a) { cur = max(x, cur + x); ans = max(ans, cur); }
    return ans;
}

int max_submatrix_sum(int n, int m, const vector<vector<int>>& mat) {
    int best = -INF;
    for (int top = 0; top < n; top++) {
        vector<int> col_sum(m, 0);
        for (int bot = top; bot < n; bot++) {
            for (int j = 0; j < m; j++) col_sum[j] += mat[bot][j];
            best = max(best, kadane(col_sum));
        }
    }
    return best;
}
```

8.3 Subset Sum via Bitset (Binary Splitting)

Determines reachable sums in $O(\frac{N\sqrt{N}}{64})$ by grouping identical weights into powers of 2.

```
bitset<1000005> reachable_subset_sums(const map<int, int>& item_counts) {
    bitset<1000005> bs;
    bs[0] = 1;
    for (auto [val, cnt] : item_counts) {
        for (int k = 1; k ≤ cnt; k ≤ 1) {
            bs |= (bs << (val * k));
            cnt -= k;
        }
        if (cnt > 0) bs |= (bs << (val * cnt));
    }
    return bs;
}
```

8.4 Interval DP (Matrix Chain Multiplication)

DP over range lengths: $dp[l][r] = \min_{l \leq k < r} (dp[l][k] + dp[k+1][r] + cost)$.

```
for (int len = 2; len ≤ n; len++) {
    for (int l = 1; l + len - 1 ≤ n; l++) {
        int r = l + len - 1;
        dp[l][r] = INF;
        for (int k = l; k < r; k++) {
            dp[l][r] = min(dp[l][r], dp[l][k] + dp[k+1][r] + cost(l, k, r));
        }
    }
}
```

8.5 Digit DP Template

Counts numbers in $[0, N]$ satisfying digit properties with memoization.

```
int dp[20][2][2][100]; // pos, tight, leading_zeros, state
string num_str;
```

```
int solve_digit(int pos, bool tight, bool lead, int state) {
    if (pos == (int)num_str.size()) return 1; // Valid number
    if (dp[pos][tight][lead][state] ≠ -1) return dp[pos][tight][lead][state];
    int limit = tight ? (num_str[pos] - '0') : 9;
    int ans = 0;
    for (int dig = 0; dig ≤ limit; dig++) {
        bool next_tight = tight && (dig == limit);
        bool next_lead = lead && (dig == 0);
        int next_state = (next_lead ? 0 : state + dig);
        ans += solve_digit(pos + 1, next_tight, next_lead, next_state);
    }
    return dp[pos][tight][lead][state] = ans;
}

int count_up_to(int n) {
    num_str = to_string(n);
    memset(dp, -1, sizeof(dp));
    return solve_digit(0, true, true, 0);
}
```

8.6 Bitmask DP (TSP / Shortest Hamiltonian Cycle)

Finds minimum weight Hamiltonian cycle visiting every vertex once in $O(2^N \cdot N^2)$.

```
int tsp(int n, const vector<vector<int>>& dist) {
    vector<vector<int>> dp(1 << n, vector<int>(n, INF));
    dp[1][0] = 0; // Start at vertex 0
    for (int mask = 1; mask < (1 << n); mask++) {
        for (int u = 0; u < n; u++) {
            if (dp[mask][u] == INF) continue;
            for (int v = 0; v < n; v++) {
                if (!(mask & (1 << v))) {
                    int nxt = mask | (1 << v);
                    dp[nxt][v] = min(dp[nxt][v], dp[mask][u] + dist[u][v]);
                }
            }
        }
    }
    int ans = INF;
    for (int u = 1; u < n; u++) ans = min(ans, dp[(1 << n) - 1][u] + dist[u][0]);
    return ans;
}
```

8.7 Tree DP (Maximum Weight Independent Set)

Chooses a subset of nodes with no two adjacent so total weight is maximized.

```
int dp[N][2]; // dp[u][0]: u not picked; dp[u][1]: u picked
void dfs_tree_dp(int u, int p, const vector<vector<int>>& g, const vector<int>& weight) {
    dp[u][0] = 0;
    dp[u][1] = weight[u];
    for (int v : g[u]) {
        if (v == p) continue;
        dfs_tree_dp(v, u, g, weight);
        dp[u][0] += max(dp[v][0], dp[v][1]);
        dp[u][1] += dp[v][0];
    }
}
```

8.8 SOS DP (Sum Over Subsets / Fast Zeta Transform)

Computes $F(mask) = \sum_{sub \subseteq mask} A(sub)$ in $O(B \cdot 2^B)$ time.

```
void sos_dp(vector<int>& f, int bits) {
    for (int b = 0; b < bits; b++) {
        for (int mask = 0; mask < (1 << bits); mask++) {
            if (mask & (1 << b)) f[mask] += f[mask ^ (1 << b)];
        }
    }
}

// Inverse Zeta Transform (Möbius transform on bitmasks):
void inverse_sos_dp(vector<int>& f, int bits) {
    for (int b = 0; b < bits; b++) {

```

```
for (int mask = 0; mask < (1 << bits); mask++) {
    if (mask & (1 << b)) f[mask] -= f[mask ^ (1 << b)];
}
}
```

8.9 Knuth's DP Optimization

Reduces $O(N^3)$ interval DP to $O(N^2)$ when optimal split point satisfies $opt[i][j - 1] \leq opt[i][j] \leq opt[i + 1][j]$.

```
void knuth_optimization(int n) {
    for (int i = 1; i <= n; i++) { opt[i][i] = i; dp[i][i] = 0; }
    for (int len = 2; len <= n; len++) {
        for (int i = 1; i + len - 1 <= n; i++) {
            int j = i + len - 1;
            dp[i][j] = INF;
            for (int k = opt[i][j - 1]; k <= min(j - 1, opt[i + 1][j]); k++) {
                int cost = dp[i][k] + dp[k + 1][j] + C(i, j);
                if (cost < dp[i][j]) { dp[i][j] = cost; opt[i][j] = k; }
            }
        }
    }
}
```

8.10 Li Chao Tree (Convex Hull Trick for Lines)

Maintains set of lines $y = mx + b$; answers minimum/maximum y at query x in $O(\log C)$.

```
struct Line { int m, b; int eval(int x) { return m * x + b; } };
struct LiChaoTree {
    struct Node { Line line; int lc = -1, rc = -1; };
    vector<Node> tree;
    LiChaoTree() { tree.push_back({{0, INF}, -1, -1}); }
    void add_line(Line nw, int node, int l, int r) {
        int mid = (l + r) / 2;
        bool left = nw.eval(l) < tree[node].line.eval(l);
        bool m = nw.eval(mid) < tree[node].line.eval(mid);
        if (m) swap(tree[node].line, nw);
        if (l == r) return;
        if (left != m) {
            if (tree[node].lc == -1) { tree[node].lc = tree.size(); tree.push_back({{0, INF}, -1, -1}); }
            add_line(nw, tree[node].lc, l, mid);
        } else {
            if (tree[node].rc == -1) { tree[node].rc = tree.size(); tree.push_back({{0, INF}, -1, -1}); }
            add_line(nw, tree[node].rc, mid + 1, r);
        }
    }
    int query(int x, int node, int l, int r) {
        if (node == -1) return INF;
        int res = tree[node].line.eval(x);
        if (l == r) return res;
        int mid = (l + r) / 2;
        if (x <= mid) return min(res, query(x, tree[node].lc, l, mid));
        return min(res, query(x, tree[node].rc, mid + 1, r));
    }
};
```

9. Greedy, Searching & Techniques

9.1 Binary Search (First True & Last True)

Standard monotonic binary search implementations preventing infinite loops.

```
// First True (Smallest x where check(x) == true):
int low = 0, high = 1e18, ans = -1;
while (low <= high) {
    int mid = low + (high - low) / 2;
    if (check(mid)) { ans = mid; high = mid - 1; }
    else low = mid + 1;
}
```

```
// Last True (Largest x where check(x) == true):
low = 0, high = 1e18, ans = -1;
while (low <= high) {
    int mid = low + (high - low) / 2;
    if (check(mid)) { ans = mid; low = mid + 1; }
    else high = mid - 1;
}
```

9.2 Floating Point Binary Search

Fixing iteration count (e.g. 100 steps) guarantees precision $2^{-100} < 10^{-30}$ without precision loops.

```
double low = 0.0, high = 1e9;
for (int iter = 0; iter < 100; iter++) {
    double mid = (low + high) / 2.0;
    if (check(mid)) high = mid;
    else low = mid;
}
```

9.3 Ternary Search (Unimodal Functions)

Finds maximum/minimum of a strictly convex/concave function in $O(\log \frac{\text{range}}{\epsilon})$.

```
// Continuous Ternary Search (Finding minimum):
double l = -1e9, r = 1e9;
for (int i = 0; i < 200; i++) {
    double m1 = l + (r - l) / 3.0;
    double m2 = r - (r - l) / 3.0;
    if (f(m1) > f(m2)) l = m1;
    else r = m2;
}
```

```
// Discrete Ternary Search (Integers):
int low = 1, high = 1e9;
while (high - low > 2) {
    int m1 = low + (high - low) / 3;
    int m2 = high - (high - low) / 3;
    if (f(m1) < f(m2)) high = m2;
    else low = m1;
}
int ans = f(low);
for (int i = low + 1; i <= high; i++) ans = min(ans, f(i));
```

9.4 Two Pointers Technique

Maintains linear relationships across two indices moving in the same or opposite directions.

```
// Two Sum in Sorted Array:
int l = 0, r = n - 1;
while (l < r) {
    int sum = a[l] + a[r];
    if (sum == target) { /* Found */ break; }
    else if (sum < target) l++;
    else r--;
}
```

9.5 Sliding Window (AtMost Trick)

Solves exact count queries: $\text{Count}(K) = \text{AtMost}(K) - \text{AtMost}(K - 1)$.

```
// Number of subarrays with at most K distinct elements:
int at_most_k_distinct(const vector<int>& a, int k) {
    if (k < 0) return 0;
    unordered_map<int, int> freq;
    int l = 0, ans = 0;
    for (int r = 0; r < (int)a.size(); r++) {
        if (freq[a[r]]++ == 0) k--;
        while (k < 0) {
            if (--freq[a[l]] == 0) k++;
            l++;
        }
        ans += (r - l + 1);
    }
    return ans;
}
```

```
int exact_k_distinct(const vector<int>& a, int k) {
    return at_most_k_distinct(a, k) - at_most_k_distinct(a, k - 1);
}
```

9.6 Meet in the Middle

Splits $N \leq 40$ search spaces into two halves of size $N/2$, then combines via sorting/binary search.

```
void generate_subset_sums(const vector<int>& a, vector<int>& sums) {
    int n = a.size();
    for (int mask = 0; mask < (1 << n); mask++) {
        int cur = 0;
        for (int i = 0; i < n; i++) if (mask & (1 << i)) cur += a[i];
        sums.push_back(cur);
    }
    sort(sums.begin(), sums.end());
}

int meet_in_the_middle(const vector<int>& a, int target) {
    int n = a.size();
    vector<int> left(a.begin(), a.begin() + n / 2);
    vector<int> right(a.begin() + n / 2, a.end());
    vector<int> sumL, sumR;
    generate_subset_sums(left, sumL);
    generate_subset_sums(right, sumR);
    int best = 0;
    for (int s : sumL) {
        for (int t : sumR) {
            if (s + t == target) {
                auto it = upper_bound(sumR.begin(), sumR.end(), target - s);
                if (it != sumR.begin()) best = max(best, s + *(--it));
            }
        }
    }
    return best;
}
```

10. Computational Geometry

10.1 Point Structure & Vector Fundamentals

2D point primitive with dot product, cross product, orientation, and distance.

```
const double EPS = 1e-9;
const double PI = acos(-1.0);
struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
    Point operator+(const Point& p) const { return Point(x + p.x, y + p.y); }
    Point operator-(const Point& p) const { return Point(x - p.x, y - p.y); }
    Point operator*(double k) const { return Point(x * k, y * k); }
    Point operator/(double k) const { return Point(x / k, y / k); }
    bool operator<(const Point& p) const {
        if (abs(x - p.x) > EPS) return x < p.x;
        return y < p.y - EPS;
    }
};

double dot(Point a, Point b) { return a.x * b.x + a.y * b.y; }
double cross(Point a, Point b) { return a.x * b.y - a.y * b.x; }
double dist_sq(Point a, Point b) { return (a.x - b.x)*(a.x - b.x) + (a.y - b.y)*(a.y - b.y); }
double dist(Point a, Point b) { return hypot(a.x - b.x, a.y - b.y); }

// Orientation: +1 = CCW, -1 = CW, 0 = Collinear
int orient(Point a, Point b, Point c) {
    double v = cross(b - a, c - a);
    if (v > EPS) return 1;
    if (v < -EPS) return -1;
    return 0;
}

Point rotate_point(Point p, double angle) {
    return Point(p.x * cos(angle) - p.y * sin(angle), p.x * sin(angle) + p.y * cos(angle));
}
```

10.2 Lines, Segments & Intersections

Segment intersection and point-to-segment distance.

```
bool on_segment(Point p, Point a, Point b) {
    return orient(a, b, p) == 0 &&
        min(a.x, b.x) ≤ p.x + EPS && p.x ≤ max(a.x, b.x) + EPS &&
        min(a.y, b.y) ≤ p.y + EPS && p.y ≤ max(a.y, b.y) + EPS;
}

bool intersect_segments(Point a, Point b, Point c, Point d) {
    int o1 = orient(a, b, c), o2 = orient(a, b, d);
    int o3 = orient(c, d, a), o4 = orient(c, d, b);
    if (o1 ≠ o2 && o3 ≠ o4) return true;
    if (on_segment(c, a, b) || on_segment(d, a, b)) return true;
    if (on_segment(a, c, d) || on_segment(b, c, d)) return true;
    return false;
}

Point line_intersection(Point a, Point b, Point c, Point d) {
    double cp = cross(b - a, d - c);
    return a + (b - a) * (cross(c - a, d - c) / cp);
}

double dist_to_segment(Point p, Point a, Point b) {
    if (dot(b - a, p - a) ≤ 0) return dist(p, a);
    if (dot(a - b, p - b) ≤ 0) return dist(p, b);
    return abs(cross(b - a, p - a)) / dist(a, b);
}
```

10.3 Polygon Area (Shoelace Formula)

Computes oriented area of polygon in $O(N)$. Vertices must be ordered CW or CCW.

```
double polygon_area(const vector<Point>& p) {
    double area = 0;
    int n = p.size();
    for (int i = 0; i < n; i++) area += cross(p[i], p[(i + 1) % n]);
    return abs(area) / 2.0;
}
```

10.4 Convex Hull (Monotonic Chain / Andrew's)

Finds vertices of the convex hull in $O(N \log N)$ counterclockwise order.

```
vector<Point> convex_hull(vector<Point> pts) {
    int n = pts.size(), k = 0;
    if (n ≤ 2) return pts;
    vector<Point> h(2 * n);
    sort(pts.begin(), pts.end());
    for (int i = 0; i < n; h[k++] = pts[i++]) {
        while (k ≥ 2 && cross(h[k - 1] - h[k - 2], pts[i] - h[k - 2]) ≤ 0) k--;
    }
    for (int i = n - 2, t = k + 1; i ≥ 0; h[k++] = pts[i--]) {
        while (k ≥ t && cross(h[k - 1] - h[k - 2], pts[i] - h[k - 2]) ≤ 0) k--;
    }
    h.resize(k - 1);
    return h;
}
```

10.5 Rotating Calipers (Polygon Diameter)

Finds the maximum distance between any two points of a convex polygon in $O(N)$.

```
double rotating_calipers_diameter(const vector<Point>& h) {
    int n = h.size();
    if (n ≤ 1) return 0;
    if (n == 2) return dist(h[0], h[1]);
    double max_d = 0;
    for (int i = 0, j = 1; i < n; i++) {
        while (cross(h[(i + 1) % n] - h[i], h[(j + 1) % n] - h[j]) > 0)
            j = (j + 1) % n;
        max_d = max({max_d, dist(h[i], h[j]), dist(h[(i + 1) % n], h[j])});
    }
    return max_d;
}
```

10.6 Point in Polygon Checks

Winding number for general polygon $O(N)$; binary search for convex polygon $O(\log N)$.

```
// General Polygon: 0 = outside, 1 = strictly inside, 2 = on boundary
int point_in_polygon(const vector<Point>& p, Point q) {
    int n = p.size(); bool inside = false;
    for (int i = 0; i < n; i++) {
        Point a = p[i], b = p[(i + 1) % n];
        if (on_segment(q, a, b)) return 2;
        if ((a.y > q.y) ≠ (b.y > q.y)) {
            double x_cross = a.x + (b.x - a.x) * (q.y - a.y) / (b.y - a.y);
            if (q.x < x_cross) inside = !inside;
        }
    }
    return inside ? 1 : 0;
}
```

10.7 Minimum Enclosing Circle (Welzl's Algorithm)

Finds smallest radius circle enclosing all points in randomized expected $O(N)$.

```
struct Circle { Point c; double r; };
Circle circumcircle_3pts(Point a, Point b, Point c) {
    Point dl = b - a, d2 = c - a;
    double d = 2.0 * cross(dl, d2);
    Point center(a.x + (dl.x*d1.x + d1.y*d1.y)*d2.y / d - (d2.x*d2.x + d2.y*d2.y)*d1.y / d,
        a.y + (d2.x*d2.x + d2.y*d2.y)*d1.x / d - (d1.x*d1.x + d1.y*d1.y)*d2.x / d);
    return {center, dist(center, a)};
}

Circle welzl(vector<Point>& pts, vector<Point> r, int n) {
    if (n == 0 || r.size() == 3) {
        if (r.empty()) return {{0, 0}, 0};
        if (r.size() == 1) return {r[0], 0};
        if (r.size() == 2) return {(r[0] + r[1]) / 2.0, dist(r[0], r[1]) / 2.0};
        return circumcircle_3pts(r[0], r[1], r[2]);
    }
    Point p = pts[n - 1];
    Circle c = welzl(pts, r, n - 1);
    if (dist(c.c, p) ≤ c.r + EPS) return c;
    r.push_back(p);
    return welzl(pts, r, n - 1);
}

Circle min_enclosing_circle(vector<Point> pts) {
    random_shuffle(pts.begin(), pts.end());
    return welzl(pts, {}, pts.size());
}
```

10.8 Classical Geometry Theorems & Formulas

Pick's theorem, cyclic quadrilaterals, incircle, and circumcircle.

- Pick's Theorem: $Area = I + \frac{B}{2} - 1$ (I : interior lattice points, B : boundary points)
- Heron's Formula: $Area = \sqrt{s(s-a)(s-b)(s-c)}$ where $s = \frac{a+b+c}{2}$
- Triangle Inradius & Circumradius: $r = \frac{Area}{s}$, $R = \frac{a \cdot b \cdot c}{4 \cdot Area}$
- Brahmagupta's Formula (Cyclic Quad): $Area = \sqrt{(s-a)(s-b)(s-c)(s-d)}$ where $s = \frac{a+b+c+d}{2}$
- Fermat Point: Minimizes distance sum to triangle vertices. If all angles $< 120^\circ$, lines to vertices meet at 120° .
- Pythagorean Triples: $a = 2st$, $b = s^2 - t^2$, $c = s^2 + t^2$ with $s > t > 0, \gcd(s, t) = 1, s \not\equiv t \pmod{2}$.

11. Bit Hacks, Game Theory & STL Reference

11.1 Bit Manipulation Cheatsheet

Common bit hacks and GCC builtins for fast binary arithmetic.

```
// Basic Bit Hacks:
x & -x // Lowest set bit (LSB) value
x & (x - 1) // Clears the lowest set bit
x | (x - 1) // Sets all trailing zero bits to 1
~x & (x + 1) // Isolates lowest 0-bit
x && !(x & (x - 1)) // Checks if x is a power of 2
(x >> i) & 1 // Gets the i-th bit (0-indexed)
x |= (1LL << i) // Sets the i-th bit
x &= ~(1LL << i) // Clears the i-th bit
x ^= (1LL << i) // Toggles the i-th bit
(1LL << k) - 1 // Mask of k lowest bits

// GCC Builtins (Append 'll' for 64-bit integers):
__builtin_popcountll(x) // Number of 1-bits
__builtin_clzll(x) // Leading zeros (Undefined for x = 0!)
__builtin_ctzll(x) // Trailing zeros (Undefined for x = 0!)
__builtin_parityll(x) // Parity of set bits (popcount % 2)
63 - __builtin_clzll(x) // MSB position (0-indexed)

// Iterate all submasks of mask m (excluding 0):
for (int s = m; s > 0; s = (s - 1) & m) { /* process s */ }

// Next number with the same popcount (GOSPER'S HACK):
int next_popcount(int x) {
    int c = x & -x, r = x + c;
    return r | (((r ^ x) >> 2) / c);
}
```

11.2 Range XOR Sum in $O(1)$

Computes $\bigoplus_{i=0}^n i$ and $\bigoplus_{i=1}^r i$ in $O(1)$.

```
int xor_0_to_n(int n) {
    if (n % 4 == 0) return n;
    if (n % 4 == 1) return 1;
    if (n % 4 == 2) return n + 1;
    return 0;
}

int range_xor(int l, int r) {
    return xor_0_to_n(r) ^ xor_0_to_n(l - 1);
}
```

11.3 Zobrist Hashing (XOR Hashing)

Assigns uniform random 64-bit integers to distinct values; hash of multiset is XOR sum of elements.

- Subarray Permutation / Anagram Check: Two ranges $[l_1, r_1]$ and $[l_2, r_2]$ have identical multiset elements iff $\bigoplus_{i=l_1}^{r_1} \text{rnd}[a[i]] == \bigoplus_{j=l_2}^{r_2} \text{rnd}[b[j]]$.
- Tree Isomorphism: Assign random value to depth or subtree sizes, hash by XORing child subtree hashes.

```
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
uint64_t random_val() { return rng(); }
```

11.4 Game Theory & Nim Rules

Impartial games under the normal play convention (last player to move wins).

- Standard Nim: Pile sizes $a_1, a_2, \dots, a_n$. First player wins iff XOR sum $X = a_1 \oplus a_2 \oplus \dots \oplus a_n \neq 0$.
- Subtraction Game: Can remove at most k stones from a pile. Equivalent to Nim with pile sizes $a_i \bmod (k + 1)$.

- Index- k Nim: Can remove from at most k piles in a single turn. Write pile sizes in binary; sum each bit column modulo $k + 1$. First player wins iff at least one bit column sum $\not\equiv 0 \pmod{k + 1}$.
- Misère Nim: Normal play, but the last player to move LOSES.
 - If all piles have size 1: First player wins iff total number of piles is EVEN.
 - If at least one pile has size > 1 : Play identically to Standard Nim ($X \neq 0 \implies$ First player wins).
- Wythoff’s Game: Two piles of size (n, m) . Can take any from one, or equal from both. Losing states are $(\lfloor k\phi \rfloor, \lfloor k\phi^2 \rfloor)$ where $\phi = \frac{1+\sqrt{5}}{2} \approx 1.6180339887$.

11.5 Sprague-Grundy Theorem

Every impartial game position is equivalent to a single Nim pile of size $g(u) = \text{mex}(\{g(v) : u \rightarrow v\})$.

```
int memo_g[N];
int grundy(int state, const vector<int>& valid_moves) {
    if (state == 0) return 0;
    if (memo_g[state] != -1) return memo_g[state];
    unordered_set<int> transitions;
    for (int m : valid_moves) {
        if (state >= m) transitions.insert(grundy(state - m, valid_moves));
    }
    int mex = 0;
    while (transitions.count(mex)) mex++;
    return memo_g[state] = mex;
}
```

11.6 STL Container Iteration & Erase

Safely deleting elements while iterating across `std::map` and `std::set` without invalidating iterators.

```
for (auto it = s.begin(); it != s.end(); ) {
    if (condition(*it)) it = s.erase(it); // Erase returns next valid iterator
    else ++it;
}
```

11.7 Direction Arrays (Grid Problems)

Standard offset deltas for 4-directional, 8-directional, and Knight moves on 2D grids.

```
// 4-Directional:
const int dx4[4] = {-1, 0, 1, 0};
const int dy4[4] = {0, 1, 0, -1};

// 8-Directional:
const int dx8[8] = {-1, -1, -1, 0, 0, 1, 1, 1};
const int dy8[8] = {-1, 0, 1, -1, 1, -1, 0, 1};

// Knight’s Moves:
const int kx[8] = {-2, -2, -1, -1, 1, 1, 2, 2};
const int ky[8] = {-1, 1, -2, 2, -2, 2, -1, 1};
```

11.8 "When to Use What" Contest Cheat Sheet

Quick problem pattern recognition table from IUPC notebook.

Problem Pattern	Algorithm / DS
Range sum with point updates	Fenwick Tree (BIT) / SegTree
Range add, Range sum/min	Lazy Segment Tree
Range minimum query (Static)	Sparse Table $O(1)$
Count pairs with XOR condition	Binary Trie / Basis Vector
Offline range distinct/frequency	Mo’s Algorithm with snake sort
Shortest path (unweighted)	BFS
Shortest path (0/1 weights)	0-1 BFS with Deque
Shortest path (positive weights)	Dijkstra with priority_queue
Shortest path (negative edges)	Bellman-Ford / SPFA
All-pairs shortest paths	Floyd-Warshall $O(V^3)$
Connected components in directed	Kosaraju / Tarjan SCC
Tree path sum/update	Heavy-Light Decomposition (HLD)
Lowest Common Ancestor	Binary Lifting $O(\log N)$
Offline subtree queries	DSU on Tree (Sack)
Pattern matching in text	KMP / Z-Algorithm / Hashing
Longest palindrome substring	Manacher’s Algorithm $O(N)$
Divisors / Prime factorization	Linear Sieve SPF $O(N)$
$a^b \pmod M$	Binary Exponentiation $O(\log b)$
Linear recurrence k -th term	Matrix Exponentiation $O(N^3 \log k)$
Optimal split interval DP	Knuth Optimization $O(N^2)$
Dynamic line min/max queries	Li Chao Tree $O(\log C)$